Day 1

Module 1

Python was created in February 20,1991
Remarkable thing about python is that it was created by 1 person Guido Van Russom
Open source, anyone can contribute to its libraries
Code readability, advanced developer productivity

Easy to Learn and Use: Python has a simple syntax, making it beginner-friendly and easy to read, write, and maintain.
Interpreted Language: Python code is executed line by line, allowing for easier debugging and faster testing.
Dynamically Typed: You don't need to declare data types; Python automatically assigns them at runtime.
Object-Oriented Programming (OOP): Python supports OOP principles like inheritance, encapsulation, and polymorphism, but it also supports procedural and functional programming.
Extensive Standard Library: Python has a vast collection of libraries and modules for various tasks, such as file I/O, regular expressions, web development, and more.
Cross-Platform: Python code can run on different operating systems like Windows, macOS, and Linux without modification.
Open Source: Python is free to use and distribute, making it accessible for all developers.
Large Community Support: Python has an active community that provides ample resources like documentation, tutorials, and third-party libraries.
Memory Management: Python has built-in garbage collection to manage memory automatically.
Embeddable and Extensible: Python can be embedded into C or C++ programs, and its functionalities can be extended using these languages.
Support for GUI Programming: Python supports the creation of graphical user interfaces through libraries like Tkinter, PyQt, or Kivy.
Integration Capabilities: Python can integrate with other languages like Java (via Jython) or .NET (via IronPython), and it supports database interaction and network programming.

Python is a versatile language used across many domains. Here are key branches of Python programming:

1. **Web Development (Back-End)**:
    - Python is popular for back-end web development. Frameworks like Django and Flask help in building server-side applications, managing databases, and handling web requests.
    - **Key Skills**: Django, Flask, REST APIs, SQL/NoSQL databases.

    - **Example**: Developing the back-end logic for a web app like Instagram or Reddit.
2. **Data Science**:
    - Python is widely used for data manipulation, analysis, and visualization. Libraries like Pandas, NumPy, and Matplotlib facilitate working with data.
    - **Key Skills**: Data wrangling, data visualization, statistics, using libraries like Pandas, Matplotlib, Seaborn.
    - **Example**: Analyzing sales data to predict trends or generate reports.
3. **Machine Learning**:
    - Python is a dominant language in machine learning (ML) and artificial intelligence (AI), using libraries like Scikit-learn, TensorFlow, and PyTorch.
    - **Key Skills**: Supervised and unsupervised learning, deep learning, NLP, using ML frameworks like TensorFlow, PyTorch.
    - **Example**: Building a recommendation system, chatbot, or image classification model.
4. **Artificial Intelligence (AI)**:
    - Python is used to develop AI applications such as natural language processing (NLP), robotics, and computer vision.
    - **Key Skills**: AI algorithms, neural networks, computer vision, NLP, reinforcement learning.
    - **Example**: Building an autonomous driving system or a voice assistant like Siri.
5. **Automation and Scripting**:
    - Python is used for scripting and automating repetitive tasks, making it a favorite for system administrators and developers.
    - **Key Skills**: Writing scripts, task automation, web scraping (with libraries like Selenium, BeautifulSoup).
    - **Example**: Automating data entry or scraping information from websites.
6. **Game Development**:
    - Python can be used to develop simple games or prototypes. Libraries like Pygame make it easy to build games and interactive applications.
    - **Key Skills**: Game loops, event handling, graphics programming with libraries like Pygame.
    - **Example**: Developing 2D games like Snake or Tetris.
7. **DevOps and Cloud**:
    - Python is used in DevOps for automation of server configurations, deployment scripts, and cloud applications.
    - **Key Skills**: Scripting with cloud services (AWS, Google Cloud), infrastructure automation, container orchestration (Docker, Kubernetes).
    - **Example**: Automating the deployment of web services or managing cloud resources.
8. **Desktop GUI Development**:
    - Python can be used to build cross-platform desktop applications with frameworks like Tkinter, PyQt, or Kivy.
    - **Key Skills**: GUI development, event-driven programming.
    - **Example**: Creating a desktop application for file management or media players.

9. **Scientific Computing**:
    - Python is used for research and scientific computing in domains like physics, chemistry, and biology. Libraries like SciPy, SymPy, and BioPython facilitate this.
    - **Key Skills**: Numerical computing, simulation, symbolic computation.
    - **Example**: Running simulations for biological research or solving differential equations.
10. **Cybersecurity**:
    - Python is frequently used to develop security tools and scripts for penetration testing, malware analysis, and network scanning.
    - **Key Skills**: Network security, cryptography, penetration testing, using libraries like Scapy and PyCrypto.
    - **Example**: Developing tools to scan networks for vulnerabilities or automate security tasks.
11. **Internet of Things (IoT)**:
    - Python is used in IoT to program microcontrollers, gather and analyze sensor data, and interact with connected devices.
    - **Key Skills**: Hardware programming, sensor interfacing, using platforms like Raspberry Pi.
    - **Example**: Building a smart home system that controls devices using Python.

Practical session:

1) Python installation
2) Anaconda Installation
3) Jupyter notebook installation
4) Using google colab for python programming

Python's syntax is designed to be clear and concise. This section will introduce essential syntax elements for writing and running Python scripts effectively.

1. Writing and Running Python Scripts

- Python scripts are saved with the `.py` extension.
- You can run a Python script by typing `python script_name.py` in the terminal or using an IDE (like Jupyter Notebook or Visual Studio Code).

```
# This is a simple Python script
print("Hello, World!")
```

2. Running Python Scripts

There are several methods to run Python scripts, depending on the environment you're using:

Method 1: Using a Terminal or Command Prompt

1. Open a terminal (on Linux/Mac) or Command Prompt (on Windows).
2. Navigate to the directory where the Python script is saved.
3. Run the script by typing:

python my_first_script.py

Method 2: Using an IDE

Most Integrated Development Environments (IDEs) allow you to run Python scripts directly by clicking a "Run" button.

- In **VS Code**, you can open the script and click the "Run" button at the top of the window.
- In **PyCharm**, you can right-click the script file and select "Run".
- **Jupyter Notebook** is also commonly used for running Python code interactively, especially in data science. In a Jupyter notebook, you can write Python code in cells and run each cell individually by pressing `Shift + Enter`.

Method 3: Running Python in an Interactive Shell

Python can also be run in an interactive mode, where you can type commands and get immediate feedback.

1. Open a terminal or command prompt.
2. Type `python` and press Enter to start the Python shell.
3. You can now type Python commands and see the output directly.

4. Running Python in Jupyter Notebooks

In Jupyter notebooks, Python code is written inside **code cells** and run interactively.

Steps:

1. Open a Jupyter Notebook.
2. Write your Python code inside a cell.
3. Press `Shift + Enter` to execute the code.

2. Comments and Docstrings

- **Single-line comments** are written using the # symbol.
- **Multi-line comments** (docstrings) are enclosed within triple quotes (''' or """).

```
# This is a single-line comment
"""
This is a multi-line comment or a docstring.
You can use it for longer explanations.
"""
```

3. Indentation in Python

Python uses indentation (spaces or tabs) to define the structure of the code. Unlike other programming languages, Python does not use curly braces ({}) or end statements.

- **Indentation** is critical in Python because it defines blocks of code.
- By default, 4 spaces are used to indent code.

```
if 5 > 2:
    print("5 is greater than 2") # This is an indented block
```

Improper indentation will result in an error:

```
if 5 > 2:
print("This will cause an IndentationError")
```

4. Variables and Assignment

- Variables in Python are dynamically typed, meaning you don't need to declare their types.
- Variable names should follow naming conventions: they must start with a letter or underscore and can contain letters, digits, and underscores.

```
x = 5
name = "Alice"
is_active = True
```

Python uses the = symbol to assign values. You can update variable values as needed:

```
x = 5
x = x + 3  # Now, x is 8
```

5. Basic Data Types

Python supports a variety of data types including:

- **Integer**: `int` (whole numbers)
- **Floating-point number**: `float` (decimal numbers)
- **String**: `str` (text)
- **Boolean**: `bool` (True/False)

```
age = 30        # int
height = 5.9            # float
name = "Alice"          # string
is_student = False   # boolean
```

6. Output with `print()`

The `print()` function is used to display output in Python.

```
name = "Alice"
print("Hello", name)
```

You can also format strings using **f-strings** (available in Python 3.6+):

```
age = 30
print(f"I am {age} years old")
```

7. Basic Input with `input()`

The `input()` function allows the user to input data. It always returns the input as a string, so type conversion is needed for other data types.

```
name = input("Enter your name: ")
print(f"Hello, {name}!")
```

For numeric input:

```
age = int(input("Enter your age: "))
print(f"You are {age} years old.")
```

8. Basic Arithmetic Operations

Python supports basic arithmetic operations like:

- **Addition** (`+`)
- **Subtraction** (`-`)
- **Multiplication** (`*`)
- **Division** (`/`)
- **Floor Division** (`//`)
- **Modulus** (`%`) for remainder
- **Exponentiation** (`**`)

```
x = 10
y = 3
print(x + y)  # 13
print(x - y)  # 7
print(x * y)  # 30
print(x / y)  # 3.3333333333333335
print(x // y) # 3 (floor division)
print(x % y)  # 1 (remainder)
print(x ** y) # 1000 (exponentiation)
```

Hands-on Exercises

1. **Simple Calculator**
   Create a basic calculator that can perform addition, subtraction, multiplication, and division using user input.

```
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

print(f"Addition: {num1 + num2}")
print(f"Subtraction: {num1 - num2}")
print(f"Multiplication: {num1 * num2}")
print(f"Division: {num1 / num2}")
```

2. Variable Manipulation
Write a program that declares three variables: an integer, a float, and a string. Perform operations to modify their values and print the updated results.

```
x = 10
y = 5.5
name = "Alice"
```

```
# Modify variables
x = x + 2
y = y * 2
name = name + " Johnson"

# Print updated values
print(x)        # 12
print(y)        # 11.0
print(name)     # Alice Johnson
```

Primitive Data Types in Python: `int`, `float`, `str`, `bool`

Python supports several built-in **primitive data types** for handling different types of data. These are fundamental data types, and Python dynamically determines their type based on the value assigned. Below are the four most common primitive data types:

---

1. Integer (`int`)

- **Description**: Used to represent whole numbers, both positive and negative, without decimals.
- **Syntax**: You simply assign a whole number to a variable, and Python will treat it as an integer.

Example:

python Copy code

```python
age = 25  # Integer
year = 2024
temperature = -5
```

Here, `age`, `year`, and `temperature` are variables of type `int`.

- **Operations**: You can perform basic arithmetic operations on integers: addition (`+`), subtraction (`-`), multiplication (`*`), division (`/`), and more.

Example:

```python
a = 10
b = 3
print(a + b)  # Output: 13
print(a - b)  # Output: 7
print(a * b)  # Output: 30
print(a // b)  # Output: 3 (Floor division)
print(a % b)  # Output: 1 (Modulus, remainder)
```

- **Type checking**: You can check the data type using the `type()` function.

```python
print(type(age))  # Output: <class 'int'>
```

2. Floating-Point Number (`float`)

- **Description**: Used to represent numbers with decimal points. Floats can also handle very large or very small numbers using scientific notation.
- **Syntax**: A float is created when you assign a number with a decimal point or use scientific notation.

Example:

```python
height = 5.9  # Float
price = 19.99
distance = 1.2e3  # Equivalent to 1200.0 (scientific notation)
```

- **Operations**: You can perform arithmetic operations on floats just like integers.

Example:

```python
x = 10.5
y = 2.3
print(x + y)  # Output: 12.8
print(x / y)  # Output: 4.565217391304348
```

- **Type checking**:

```python
print(type(height))  # Output: <class 'float'>
```

Note: Division between two integers always results in a float.

```python
print(5 / 2)  # Output: 2.5
```

3. String (`str`)

- **Description**: Used to represent text (a sequence of characters). Strings are enclosed within either single quotes (`'`) or double quotes (`"`).
- **Syntax**: Assign a value within quotes to create a string.

Example:

python Copy code

```python
name = "Alice"  # String
city = 'New York'
```

- **Operations**:
  - You can concatenate (join) strings using the `+` operator.
  - Use `len()` to get the length of the string.
  - Access individual characters in a string using indexing (starting at 0).

Example:

```python
first_name = "John"
last_name = "Doe"
full_name = first_name + " " + last_name  # Concatenation
print(full_name)  # Output: John Doe

# Accessing characters in the string
print(first_name[0])  # Output: J (first character)
print(first_name[-1])  # Output: n (last character)

# String length
print(len(first_name))  # Output: 4
```

- **Type checking**:

```python
print(type(name))  # Output: <class 'str'>
```

4. Boolean (`bool`)

- **Description**: Used to represent truth values. A boolean can only be `True` or `False`. These are particularly useful in conditional expressions.
- **Syntax**: Assign either `True` or `False` to a variable.

Example:

```python
is_student = True
has_passed = False
```

- **Boolean operations**: You can use logical operators such as `and`, `or`, and `not` with boolean values.

Example:

```python
is_raining = True
is_sunny = False

print(is_raining and is_sunny)  # Output: False
print(is_raining or is_sunny)   # Output: True
print(not is_raining)           # Output: False
```

- **Type checking**:

```python
print(type(is_student))  # Output: <class 'bool'>
```

Hands-on Exercises

1. **Integer and Float Operations** Write a Python program that performs different operations on integers and floats and prints the results.

```python
a = 15
b = 4.5

print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
```

2. **String Manipulation** Create a program that takes a user's first and last name and prints a greeting.

python Copy code

```python
first_name = input("Enter your first name: ")
last_name = input("Enter your last name: ")
print(f"Hello, {first_name} {last_name}!")
```

3. **Boolean Conditions** Write a Python script that checks if a user is both a student and has passed a course.

python Copy code

```python
is_student = True
has_passed = False

if is_student and has_passed:
    print("You can proceed to the next course.")
else:
    print("You need to pass the course first.")
```

Complex Data Types in Python: Lists, Tuples, Sets, and Dictionaries

Python provides several built-in **complex data types** that allow you to store and manipulate collections of values efficiently. Each complex data type has its own characteristics and use cases.

---

1. Lists

- **Description**: Lists are ordered, mutable (modifiable), and allow duplicate elements. They are one of the most versatile data types in Python.
- **Syntax**: Lists are defined using square brackets `[]`, with elements separated by commas.

Basic Example:

```python
fruits = ["apple", "banana", "cherry"]
```

Key Operations:

- **Accessing elements**: Use indexing to access individual elements.

```python
print(fruits[0])  # Output: apple
```

- **Negative indexing**: Start from the end of the list.

```python
print(fruits[-1])  # Output: cherry
```

- **Slicing**: Retrieve a subset of the list.

```python
print(fruits[0:2])  # Output: ['apple', 'banana']
```

- **Appending elements**: Add new elements at the end of the list.

```python
fruits.append("orange")
print(fruits)  # Output: ['apple', 'banana', 'cherry', 'orange']
```

- **Inserting elements**: Insert at a specific position.

```python
fruits.insert(1, "grape")
print(fruits)  # Output: ['apple', 'grape', 'banana', 'cherry']
```

- **Removing elements**:
  - Use `remove()` to delete by value.
  - Use `pop()` to remove by index (default is the last element).

```python
fruits.remove("banana")
fruits.pop()  # Removes the last element by default
```

Advanced List Operations:

- **List comprehension**: A concise way to create lists.

```python
squares = [x**2 for x in range(1, 6)]
print(squares)  # Output: [1, 4, 9, 16, 25]
```

- **Sorting**: Sort a list in place using `sort()` or create a sorted copy using `sorted()`.

```python
numbers = [5, 2, 9, 1]
numbers.sort()
print(numbers)  # Output: [1, 2, 5,
```

- **Copying a list**:
  - Use `copy()` to create a shallow copy.
  - Use slicing (`[:]`) to copy the list.

```python
new_list = fruits.copy()
another_list = fruits[:]
```

- **List concatenation**: Combine lists using `+`.

```python
list1 = [1, 2, 3]
list2 = [4, 5, 6]
combined = list1 + list2  # Output: [1, 2, 3, 4, 5, 6]
```

2. Tuples

- **Description**: Tuples are ordered, immutable (cannot be modified after creation), and can contain duplicate elements. Tuples are often used for data that should not change.
- **Syntax**: Tuples are defined using parentheses `()`.

Basic Example:

```python
coordinates = (10, 20)
```

Key Operations:

- **Accessing elements**: Just like lists, you can use indexing to access tuple elements.

```python
print(coordinates[0])  # Output: 10
```

- **Negative indexing and slicing**: Similar to lists.

```python
print(coordinates[-1])  # Output: 20
```

- **Unpacking tuples**: Assign tuple values directly to variables.

```python
x, y = coordinates
print(x)  # Output: 10
```

- **Concatenating tuples**:

```python
tuple1 = (1, 2, 3)
tuple2 = (4, 5, 6)
combined = tuple1 + tuple2  # Output: (1, 2, 3, 4, 5, 6)
```

- **Immutability**: Once created, you cannot change elements within a tuple. To modify, you need to create a new tuple.

Advanced Tuple Operations:

- **Tuple comprehension**: There is no direct tuple comprehension like in lists, but you can use the `tuple()` constructor with a generator expression.

```python
squares = tuple(x**2 for x in range(1, 6))
print(squares)  # Output: (1, 4, 9, 16, 25)
```

- **Tuples as dictionary keys**: Since tuples are immutable, they can be used as keys in dictionaries (unlike lists).

```python
points = {(1, 2): "A", (3, 4): "B"}
print(points[(1, 2)])  # Output: A
```

3. Sets

- **Description**: Sets are unordered, mutable, and do not allow duplicate elements. They are useful for membership testing and eliminating duplicate values.
- **Syntax**: Sets are defined using curly braces `{}` or the `set()` function.

Basic Example:

```python
unique_numbers = {1, 2, 3, 4}
```

Key Operations:

- **Adding elements**: Add elements using `add()`.

```python
unique_numbers.add(5)
```

- **Removing elements**:
  - Use `remove()` to remove by value (raises error if not found).
  - Use `discard()` to remove without error if not found.

```python
unique_numbers.remove(3)
unique_numbers.discard(10)  # Does not raise an error if 10 is not present
```

- **Set operations**: Perform common set operations like union, intersection, and difference.

```python
set_a = {1, 2, 3}
set_b = {3, 4, 5}

union = set_a | set_b  # {1, 2, 3, 4, 5}
intersection = set_a & set_b  # {3}
difference = set_a - set_b  # {1, 2}
```

- **Membership testing**:

```python
print(2 in set_a)  # Output: True
```

Advanced Set Operations:

- **Set comprehension**: Similar to list comprehension, but creates a set.

```python
squares = {x**2 for x in range(1, 6)}
print(squares)  # Output: {1, 4, 9, 16, 25}
```

- **Frozen sets**: Immutable sets that cannot be modified once created.

```python
frozen_set = frozenset([1, 2, 3])
```

---

4. Dictionaries

- **Description**: Dictionaries store data as key-value pairs. They are unordered, mutable, and keys must be unique and immutable (typically strings or numbers).

- **Syntax**: Dictionaries are defined using curly braces `{}` with key-value pairs separated by colons (`:`).

Basic Example:

```python
student = {"name": "Alice", "age": 25, "major": "Physics"}
```

Key Operations:

- **Accessing elements**: Retrieve values using keys.

```python
print(student["name"])  # Output: Alice
```

- **Adding/updating elements**: Add new key-value pairs or update existing ones.

```python
student["age"] = 26  # Updates age
student["GPA"] = 3.8  # Adds new key-value pair
```

- **Removing elements**: Use `del` or `pop()` to remove elements by key.

```python
del student["major"]
student.pop("GPA")
```

- **Dictionary methods**:
  - `keys()`: Returns a view of the dictionary's keys.
  - `values()`: Returns a view of the dictionary's values.
  - `items()`: Returns key-value pairs as tuples.

```python
print(student.keys())   # Output: dict_keys(['name', 'age'])
print(student.values()) # Output: dict_values(['Alice', 26])
print(student.items())  # Output: dict_items([('name', 'Alice'), ('age', 26)])
```

Advanced Dictionary Operations:

- **Dictionary comprehension**: Create a dictionary using a concise expression.

```python
squares = {x: x**2 for x in range(1, 6)}
print(squares)  # Output: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

- **Merging dictionaries**: Combine two dictionaries using the `update()` method or the `{**dict1, **dict2}` syntax.

```python
dict1 = {"a": 1, "b": 2}
dict2 = {"b": 3, "c": 4}
dict1.update(dict2)
print(dict1)  # Output: {'a': 1, 'b': 3, 'c': 4}
```

Hands-on Exercises

1. **Lists and Tuples**: Write a program that manipulates a list of students' names and then converts it into a tuple.

```python
students = ["Alice", "Bob", "Charlie"]
students.append("Diana")
print(students)  # Output
```

Day 2

Dictionary

Create an empty dictionary and populate data into it

```
# Create a dictionary
dictionary = {} # Curly braces method
another_dictionary = dict() # Dict method

# Are the above dictionaries equivalent?
print(type(dictionary))
print(type(another_dictionary))
print(dictionary == another_dictionary)
```

```
# Populate the dictionary
dictionary["key1"] = "value1"

# Access key1
print(dictionary["key1"])
```

Create a prepopulated dictionary using syntax below

```
# Create a dictionary with preinserted keys/values
dictionary = {"key1": "value1"}

# Access key1
print(dictionary["key1"])
```

We can also create a dict(), in which we supply keys and values as a keyword argument list or as a list of tuples

```
# Keyword argument list
dictionary = dict(key1="value1", key2="value2")

# Display the dictionary
print(dictionary)
```

```
# List of tuples
dictionary = dict([("key1", "value1"), ("key2", "value2")])

# Display the dictionary
print(dictionary)
```

```
# Add more elements to the dictionary
dictionary[42] = "the answer to the ultimate question of life, the universe, and everythi
dictionary[1.2] = ["one point two"]
dictionary["list"] = ["just", "a", "list", "with", "an", "integer", 3]

# Display the dictionary
print(dictionary)
```

```
# Modify a value
dictionary["list"] = ["it's another", "list"]

# Display the dictionary
print(dictionary)
print()

# Access the value of "list"
print(dictionary["list"])
```

updated() methods

```
# Create a Harry Potter dictionary
harry_potter_dict = {
    "Harry Potter": "Gryffindor",
    "Ron Weasley": "Gryffindor",
    "Hermione Granger": "Gryffindor"
}

# Display the dictionary
print(harry_potter_dict)
```

```
# Characters to add to the Harry Potter dictionary
add_characters_1 = {
    "Albus Dumbledore": "Gryffindor",
    "Luna Lovegood": "Ravenclaw"
}

# Merge dictionaries
harry_potter_dict.update(add_characters_1)

# Display the dictionary
print(harry_potter_dict)
```

```
# Use iterables to update a dictionary
add_characters_2 = [
    ["Draco Malfoy", "Slytherin"],
    ["Cedric Diggory", "Hufflepuff"]
]
harry_potter_dict.update(add_characters_2)

print(harry_potter_dict)
```

del() function

```
# Delete a key:value pair
del harry_potter_dict["Minerva McGonagall"]

print(harry_potter_dict)
```

If we're trying to delete a pair that isn't present in the dictionary, we'll get a KeyError:

```
# Delete a key:value pair that doesn't exist in the dictionary
del harry_potter_dict["Voldemort"]
```

```
---------------------------------------------------------------------------

KeyError                                  Traceback (most recent call last)

~\AppData\Local\Temp/ipykernel_10024/3860415884.py in
      1 # Delete a key:value pair that doesn't exist in the dictionary
----> 2 del harry_potter_dict["Voldemort"]

KeyError: 'Voldemort'
```

popitem(): deletes the last item inserted into the dictionary

```
# Insert Voldemort
harry_potter_dict["Voldemort"] = "Slytherin"

print("Dictionary with Voldemort:")
print(harry_potter_dict)
print()

# Remove the last inserted item (Voldemort)
harry_potter_dict.popitem()

print("Dictionary after popping the last inserted item (Voldemort):")
print(harry_potter_dict)
```

get() method

If we try to access a value of the key that doesn't exist in the dictionary, Python will return a `KeyError`. To get around this problem, we can use the `get()` method that will return the value if its key is in the dictionary, or it will return some default value that we set:

```
# Return an existing value
print(harry_potter_dict.get("Harry Potter", "Key not found"))

# Return a default value if no key found in the dictionary
print(harry_potter_dict.get("Voldemort", "Key not found"))

# Try to retrieve a value of a non-existing key without get
print(harry_potter_dict["Voldemort"])
```

items(), *keys()*, and *values()*

What if we want to return all the `key:value` pairs? Or just the keys? What about the values?

The answer to the first question is the `items()` method. When used on a dictionary, it will return a `dict_items` object, which is essentially a list of tuples, each containing a key and a value. This method may be useful when we loop through a dictionary as we'll see later.

```
print(harry_potter_dict.items())
```

```
dict_items([('Harry Potter', 'Gryffindor'), ('Ron Weasley', 'Gryffindor'), ('Hermione
```

If we want to get just the keys, we should use the `keys()` method. It will return a `dict_keys` object:

```
print(harry_potter_dict.keys())
```

```
dict_keys(['Harry Potter', 'Ron Weasley', 'Hermione Granger', 'Albus Dumbledore', 'Lu
```

Finally, we have the `values()` method that will return the values as a `dict_values` object:

```
print(harry_potter_dict.values())
```

```
dict_values(['Gryffindor', 'Gryffindor', 'Gryffindor', 'Gryffindor', 'Ravenclaw', 'Sl
```

Looping through dictionary

```
for key, value in harry_potter_dict.items():
    print((key, value))
```

```
('Harry Potter', 'Gryffindor')
('Ron Weasley', 'Gryffindor')
('Hermione Granger', 'Gryffindor')
('Albus Dumbledore', 'Gryffindor')
('Luna Lovegood', 'Ravenclaw')
('Draco Malfoy', 'Slytherin')
('Cedric Diggory', 'Hufflepuff')
('Rubeus Hagrid', 'Gryffindor')
('Voldemort', 'Slytherin')
```

```
# Alternatively
for key_value in harry_potter_dict.items():
    print(key_value)
```

```
('Harry Potter', 'Gryffindor')
('Ron Weasley', 'Gryffindor')
('Hermione Granger', 'Gryffindor')
('Albus Dumbledore', 'Gryffindor')
('Luna Lovegood', 'Ravenclaw')
('Draco Malfoy', 'Slytherin')
('Cedric Diggory', 'Hufflepuff')
('Rubeus Hagrid', 'Gryffindor')
('Voldemort', 'Slytherin')
```

```python
for key, value in harry_potter_dict.items():
    print(f"The current key is {key} and its value is {value}.")
```

```
The current key is Harry Potter and its value is Gryffindor.
The current key is Ron Weasley and its value is Gryffindor.
The current key is Hermione Granger and its value is Gryffindor.
The current key is Albus Dumbledore and its value is Gryffindor.
The current key is Luna Lovegood and its value is Ravenclaw.
The current key is Draco Malfoy and its value is Slytherin.
The current key is Cedric Diggory and its value is Hufflepuff.
The current key is Rubeus Hagrid and its value is Gryffindor.
The current key is Voldemort and its value is Slytherin.
```

```python
# Loop only through the keys
for key in harry_potter_dict.keys():
    print(key)
```

```
Harry Potter
Ron Weasley
Hermione Granger
Albus Dumbledore
Luna Lovegood
Draco Malfoy
Cedric Diggory
Rubeus Hagrid
Voldemort
```

```python
# Loop only through the values
for value in harry_potter_dict.values():
    print(value)
```

```
Gryffindor
Gryffindor
Gryffindor
Gryffindor
Ravenclaw
Slytherin
Hufflepuff
Gryffindor
Slytherin
```

```python
from collections import Counter

# Frequency of values
counter = Counter(harry_potter_dict.values())
print(counter)
```

```
Counter({'Gryffindor': 5, 'Slytherin': 2, 'Ravenclaw': 1, 'Hufflepuff': 1})
```

The returned object `Counter` is actually very similar to a dictionary. We can use the `keys()`, `values()`, and `items()` methods on it!

```
# Items of Counter
for k, v in counter.items():
    print((k, v))
```

```
('Gryffindor', 5)
('Ravenclaw', 1)
('Slytherin', 2)
('Hufflepuff', 1)
```

```
# Keys of Counter
for k in counter.keys():
    print(k)
```

```
Gryffindor
Ravenclaw
Slytherin
Hufflepuff
```

```
# Values of Counter
for f in counter.values():
    print(f)
```

```
5
1
2
1
```

4. Nested Dictionaries

- **Task**: Have students create a nested dictionary to represent information such as a classroom with students, where each student has their own dictionary containing attributes like age and grade.
- **Example**:

python Copy code

```
classroom = {
    'student1': {'name': 'Alice', 'age': 10, 'grade': 'A'},
    'student2': {'name': 'Bob', 'age': 11, 'grade': 'B'}
}
```

Arithmetic operations

```
a = 7
b = 2

# addition
print ('Sum: ', a + b)

# subtraction
print ('Subtraction: ', a - b)

# multiplication
print ('Multiplication: ', a * b)

# division
print ('Division: ', a / b)

# floor division
print ('Floor Division: ', a // b)

# modulo
print ('Modulo: ', a % b)

# a to the power b
print ('Power: ', a ** b)
```

Assignment operators

Operator	Name	Example
`=`	Assignment Operator	`a = 7`
`+=`	Addition Assignment	`a += 1 # a = a + 1`
`-=`	Subtraction Assignment	`a -= 3 # a = a - 3`
`*=`	Multiplication Assignment	`a *= 4 # a = a * 4`
`/=`	Division Assignment	`a /= 3 # a = a / 3`
`%=`	Remainder Assignment	`a %= 10 # a = a % 10`
`**=`	Exponent Assignment	`a **= 10 # a = a ** 10`

```
# assign 10 to a
a = 10

# assign 5 to b
b = 5

# assign the sum of a and b to a
a += b      # a = a + b

print(a)

# Output: 15
```

Comparison Operators

Operator	Meaning	Example
`==`	Is Equal To	`3 == 5` gives us `False`
`!=`	Not Equal To	`3 != 5` gives us `True`
`>`	Greater Than	`3 > 5` gives us `False`
`<`	Less Than	`3 < 5` gives us `True`
`>=`	Greater Than or Equal To	`3 >= 5` give us `False`
`<=`	Less Than or Equal To	`3 <= 5` gives us `True`

```
a = 5

b = 2

# equal to operator
print('a == b =', a == b)

# not equal to operator
print('a != b =', a != b)

# greater than operator
print('a > b =', a > b)

# less than operator
print('a < b =', a < b)

# greater than or equal to operator
print('a >= b =', a >= b)

# less than or equal to operator
print('a <= b =', a <= b)
```

Python logical operators

Operator	Example	Meaning
`and`	a **and** b	**Logical AND:** `True` only if both the operands are `True`
`or`	a **or** b	**Logical OR:** `True` if at least one of the operands is `True`
`not`	**not** a	**Logical NOT:** `True` if the operand is `False` and vice-versa.

```
# logical AND
print(True and True)     # True
print(True and False)    # False

# logical OR
print(True or False)     # True

# logical NOT
print(not True)          # False
```

Python bitwise operators

In the table below: Let `x` = **10** (`0000 1010` in binary) and `y` = **4** (`0000 0100` in binary)

Operator	Meaning	Example
`&`	Bitwise AND	x & y = 0 (`0000 0000`)
`|`	Bitwise OR	x | y = 14 (`0000 1110`)
`~`	Bitwise NOT	~x = -11 (`1111 0101`)
`^`	Bitwise XOR	x ^ y = 14 (`0000 1110`)

Identity operators

```
x1 = 5
y1 = 5
x2 = 'Hello'
y2 = 'Hello'
x3 = [1,2,3]
y3 = [1,2,3]

print(x1 is not y1)  # prints False

print(x2 is y2)  # prints True

print(x3 is y3)  # prints False
```

Membership operators

```
message = 'Hello world'
dict1 = {1:'a', 2:'b'}

# check if 'H' is present in message string
print('H' in message)  # prints True

# check if 'hello' is present in message string
print('hello' not in message)  # prints True

# check if '1' key is present in dict1
print(1 in dict1)  # prints True

# check if 'a' key is present in dict1
print('a' in dict1)  # prints False
```

Conditional Statements

Python if Statement

An `if` statement executes a block of code only when the specified condition is met.

Syntax

```
if condition:
    # body of if statement
```

Here, **condition** is a boolean expression, such as `number > 5`, that evaluates to either `True` or `False`.

- If `condition` evaluates to `True`, the body of the `if` statement is executed.
- If `condition` evaluates to `False`, the body of the `if` statement will be skipped from execution.

Example: Python if Statement

```
number = int(input('Enter a number: '))

# check if number is greater than 0
if number > 0:
    print(f'{number} is a positive number.')

print('A statement outside the if statement.')
```

Run Code »

Indentation in Python

Python uses indentation to define a block of code, such as the body of an `if` statement. For example,

```
x = 1
total = 0

# start of the if statement
if x != 0:
    total += x
    print(total)
# end of the if statement

print("This is always executed.")
```

Example: Python if...else Statement

```
number = int(input('Enter a number: '))

if number > 0:
    print('Positive number')
else:
    print('Not a positive number')

print('This statement always executes')
```

Run Code »

Example: Python if...elif...else Statement

```
number = -5

if number > 0:
    print('Positive number')

elif number < 0:
    print('Negative number')

else:
    print('Zero')

print('This statement is always executed')
```

Python nested if statements

```
number = 5

# outer if statement
if number >= 0:
    # inner if statement
    if number == 0:
      print('Number is 0')

    # inner else statement
    else:
        print('Number is positive')

# outer else statement
else:
    print('Number is negative')
```

For loop

In Python, we use a `for` loop to iterate over sequences such as lists, strings, dictionaries, etc.

```
languages = ['Swift', 'Python', 'Go']

# access elements of the list one by one
for lang in languages:
    print(lang)
```

Indentation in Loop

In Python, we use indentation to define a block of code, such as the body of a loop. For example,

```
languages = ['Swift', 'Python', 'Go']

# Start of loop
for lang in languages:
    print(lang)
    print('-----')
# End of for loop

print('Last statement')
```

Example: Loop Through a String

```
language = 'Python'

# iterate over each character in language
for x in language:
    print(x)
```

for Loop with Python range()

In Python, the range() function returns a sequence of numbers. For example,

```
values = range(4)
```

Here, `range(4)` returns a sequence of **0**, **1**, **2** ,and **3**.

Since the `range()` function returns a sequence of numbers, we can iterate over it using a `for` loop. For example,

```
# iterate from i = 0 to i = 3
for i in range(4):
    print(i)
```

Task: Print a multiplication table of user input number

```
1 num=int(input("Enter number"))
2 for i in range(1,11):
3   print(f"{num} x {i} = {int(num*i)}")
```

```
Enter number6
```

While loop

In Python, we use a `while` loop to repeat a block of code until a certain condition is met. For example,

```
number = 1

while number <= 3:
    print(number)
    number = number + 1
```

Run Code »

Example: Python while Loop

```
# Print numbers until the user enters 0
number = int(input('Enter a number: '))

# iterate until the user enters 0
while number != 0:
    print(f'You entered {number}.')
    number = int(input('Enter a number: '))

print('The end.')
```

Run Code »

Infinite while Loop

If the condition of a `while` loop always evaluates to `True`, the loop runs continuously, forming an **infinite while loop**. For example,

```
age = 32

# The test condition is always True
while age > 18:
    print('You can vote')
```

Task: Multiplication table using while loop

```python
num=int(input("Enter number"))
i=1
while (i<=10):
  print(f"{num} x {i} = {int(num*i)}")
  i+=1
```

Loop control (break and continue)

Example: break Statement with for Loop

We can use the `break` statement with the `for` loop to terminate the loop when a certain condition is met. For example,

```python
for i in range(5):
    if i == 3:
        break
    print(i)
```

Run Code »

Output

```
0
1
2
```

Example: continue Statement with for Loop

We can use the `continue` statement with the `for` loop to skip the current iteration of the loop and jump to the next iteration. For example,

```
for i in range(5):
    if i == 3:
        continue
    print(i)
```

Run Code »

Output

```
0
1
2
4
```

1. **`else` in loops**: Python loops (`for` and `while`) can have an `else` clause, which runs only if the loop completes without encountering a `break`. It won't execute if the loop is exited early.

```python
for i in range(5):
    if i == 3:
        break
else:
    print("No break occurred")  # This won't print since 'break' was used
```

3. **Negative step in `range()`**: You can use a negative step in `range()` to iterate in reverse order.

```python
for i in range(5, 0, -1):
    print(i)  # Output: 5, 4, 3, 2, 1
```

4. `enumerate()` **gives both index and value**: Instead of manually keeping track of indices, you can use `enumerate()` to get the index and value of each item in an iterable at the same time.

```python
for index, value in enumerate(['a', 'b', 'c']):
    print(index, value)
# Output: 0 a, 1 b, 2 c
```

5. **You can iterate over multiple iterables simultaneously with** `zip()`: Python's `zip()` function allows you to loop through two or more iterables in parallel.

```python
for a, b in zip([1, 2, 3], ['a', 'b', 'c']):
    print(a, b)  # Output: 1 a, 2 b, 3 c
```

6. **Loop variables remain in scope**: After a loop ends, the loop variable is still accessible and retains its last assigned value, unlike some other languages where the variable is scoped to the loop.

```python
for i in range(3):
    pass
print(i)  # Output: 2
```

Task: Write a Python program that takes a list of numbers as input and counts how many of them are even and how many are odd. The program should use a loop to iterate over the numbers and conditional statements to check if a number is even or odd.

```
data=[5,4,7,2,6,9]
even_odd = ['Even' if i % 2 == 0 else 'Odd' for i in data]
print(even_odd)
```

Questions

Task: Ask students to write a program that takes a list of student scores and classifies each score into letter grades (A, B, C, D, F) based on defined thresholds. Use a loop to iterate through the scores and a series of conditional statements to classify the grade.

```
scores = [95, 82, 74, 67, 59, 88, 93, 45]

grades = [
    'A' if score >= 90 else
    'B' if score >= 80 else
    'C' if score >= 70 else
    'D' if score >= 60 else
    'F'
    for score in scores
]

print(grades)
```

```
['A', 'B', 'C', 'D', 'F', 'B', 'A', 'F']
```

Task: Write a Python program that checks if a number is prime. The program should prompt the user to input a number and use a loop to check if the number has any divisors other than 1 and itself. Use conditional statements inside the loop to determine if the number is prime.

Defining functions

Calling a Function

In the above example, we have declared a function named `greet()`.

```
def greet():
    print('Hello World!')
```

```
greet()
```

Python Function Arguments

Arguments are inputs given to the function.

```
def greet(name):
    print("Hello", name)

# pass argument
greet("John")
```

Sample Output 1

```
Hello John
```

```python
# function with two arguments
def add_numbers(num1, num2):
    sum = num1 + num2
    print("Sum: ", sum)

# function call with two values
add_numbers(5, 4)
```

Output

```
Sum: 9
```

Use case can be reused multiple times with multiple parameters

The return Statement

We return a value from the function using the `return` statement.

```
# function definition
def find_square(num):
    result = num * num
    return result

# function call
square = find_square(3)

print('Square:', square)
```

Output

```
Square: 9
```

The pass Statement

The `pass` statement serves as a placeholder for future code, preventing errors from empty code blocks.

It's typically used where code is planned but has yet to be written.

```
def future_function():
    pass

# this will execute without any action or error
future_function()
```

Functions can have inner functions

```
def outer_func(x):
    def inner_func(y):
        return x + y
    return inner_func

add_five = outer_func(5)
print(add_five(10))  # Output: 15
```

Python Library Functions

Python provides some built-in functions that can be directly used in our program.

We don't need to create the function, we just need to call them.

Some Python library functions are:

1. print() - prints the string inside the quotation marks
2. `sqrt()` - returns the square root of a number
3. pow() - returns the power of a number

These library functions are defined inside the module. And to use them, we must include the module inside our program.

For example, `sqrt()` is defined inside the math module.

Example: Python Library Function

```
import math

# sqrt computes the square root
square_root = math.sqrt(4)

print("Square Root of 4 is",square_root)

# pow() comptes the power
power = pow(2, 3)

print("2 to the power 3 is",power)
```

Run Code »

Output

```
Square Root of 4 is 2.0
2 to the power 3 is 8
```

User Defined Function Vs Standard Library Functions

In Python, functions are divided into two categories: user-defined functions and standard library functions. These two differ in several ways:

User-Defined Functions

These are the functions we create ourselves. They're like our custom tools, designed for specific tasks we have in mind.

They're not part of Python's standard toolbox, which means we have the freedom to tailor them exactly to our needs, adding a personal touch to our code.

Standard Library Functions

Think of these as Python's pre-packaged gifts. They come built-in with Python, ready to use.

These functions cover a wide range of common tasks such as mathematics, file operations, working with strings, etc.

They've been tried and tested by the Python community, ensuring efficiency and reliability.

Default Arguments in Functions

Python allows functions to have default argument values. Default arguments are used when no explicit values are passed to these parameters during a function call.

Let's look at an example.

```
def greet(name, message="Hello"):
    print(message, name)

# calling function with both arguments
greet("Alice", "Good Morning")

# calling function with only one argument
greet("Bob")
```

Run Code »

Output

```
Good Morning Alice
```

1. What They Stand For

- `*args` collects **positional arguments** into a tuple.
- `**kwargs` collects **keyword arguments** into a dictionary.

python Copy code

```python
def example(*args, **kwargs):
    print(args)   # Tuple of positional arguments
    print(kwargs) # Dictionary of keyword arguments

example(1, 2, 3, a=4, b=5)
# Output:
# (1, 2, 3)
# {'a': 4, 'b': 5}
```

3. Order Matters

When combining `*args`, `**kwargs`, and regular arguments, the order in the function definition must be:

- Regular positional arguments
- `*args` for additional positional arguments
- Regular keyword arguments
- `**kwargs` for additional keyword arguments.

python Copy code

```python
def func(a, *args, b=2, **kwargs):
    print(a, args, b, kwargs)

func(1, 3, 4, 5, b=7, x=10, y=20)
# Output: 1 (3, 4, 5) 7 {'x': 10, 'y': 20}
```

4. Mixing with Regular Arguments

- You can still have regular arguments when using `*args` and `**kwargs`. Regular arguments must always appear before `*args`, and keyword-only arguments must appear after `**kwargs`.

python Copy code

```python
def example(a, *args, b, **kwargs):
    print(a, args, b, kwargs)

example(1, 2, 3, b=4, x=5)
# Output: 1 (2, 3) 4 {'x': 5}
```

5. `*args` and `**kwargs` Can Be Empty

- Both `*args` and `**kwargs` are optional. If not passed, they simply default to an empty tuple and dictionary, respectively.

```python
def example(*args, **kwargs):
    print(args)
    print(kwargs)

example()  # Output: () {}
```

7. Chaining Functions with `*args` and `**kwargs`

- You can use `*args` and `**kwargs` to pass arguments between functions, especially when chaining multiple functions together. This is useful when you don't know the exact number of arguments a function may need to pass to another.

```python
def func1(*args, **kwargs):
    func2(*args, **kwargs)

def func2(x, y, z=3):
    print(x, y, z)

func1(1, 2, z=4)  # Output: 1 2 4
```

Default Arguments in Functions

Using `*args` and `**kwargs` in Functions

We can handle an arbitrary number of arguments using special symbols `*args` and `**kwargs`.

`*args` in Functions

Using `*args` allows a function to take any number of positional arguments.

```
# function to sum any number of arguments
def add_all(*numbers):
    return sum(numbers)

# pass any number of arguments
print(add_all(1, 2, 3, 4))
```

Run Code »

Output

```
10
```

`*kwargs` in Functions

Using `**kwargs` allows the function to accept any number of keyword arguments.

```
# function to print keyword arguments
def greet(**words):
    for key, value in words.items():
        print(f"{key}: {value}")

# pass any number of keyword arguments
greet(name="John", greeting="Hello")
```

Output

```
name: John
greeting: Hello
```

Example of a recursive function

```
def factorial(x):
    """This is a recursive function
    to find the factorial of an integer"""

    if x == 1:
        return 1
    else:
        return (x * factorial(x-1))


num = 3
print("The factorial of", num, "is", factorial(num))
```

5. Indirect Recursion (Mutual Recursion)

- Functions can call each other recursively, a concept known as **indirect recursion** or **mutual recursion**. It involves two or more functions calling each other in a loop.

python Copy code

```python
def is_even(n):
    if n == 0:
        return True
    return is_odd(n - 1)

def is_odd(n):
    if n == 0:
        return False
    return is_even(n - 1)

print(is_even(10))  # Output: True
print(is_odd(10))   # Output: False
```

Scope of variable (local vs global)

Python Program to Find Sum of Natural Numbers Using Recursion
Python Program to Display Fibonacci Sequence Using Recursion

Lambda function

In Python, a lambda function is a special type of function without the function name. For example,

Let's see an example,

```
greet = lambda : print('Hello World')
```

Here, we have defined a lambda function and assigned it to the variable named `greet`.

To execute this lambda function, we need to call it. Here's how we can call the lambda function

```
# call the lambda
greet()
```

Example: Python Lambda Function

```
# declare a lambda function
greet = lambda : print('Hello World')

# call lambda function
greet()

# Output: Hello World
```

```
# lambda that accepts one argument
greet_user = lambda name : print('Hey there,', name)

# lambda call
greet_user('Delilah')

# Output: Hey there, Delilah
```

Modules in python

As our program grows bigger, it may contain many lines of code. Instead of putting everything in a single file, we can use modules to separate codes in separate files as per their functionality. This makes our code organized and easier to maintain.

Module is a file that contains code to perform a specific task. A module may contain variables, functions, classes etc. Let's see an example,

Let us create a module. Type the following and save it as `example.py`.

```
# Python Module addition

def add(a, b):

   result = a + b
   return result
```

```
import example
```

This does not import the names of the functions defined in `example` directly in the current symbol table. It only imports the module name `example` there.

Using the module name we can access the function using the dot `.` operator. For example:

```
example.add(4,5) # returns 9
```

Import Python Standard Library Modules

The Python standard library contains well over **200** modules. We can import a module according to our needs.

Suppose we want to get the value of `pi`, first we import the math module and use `math.pi`. For example,

```
# import standard math module
import math

# use math.pi to get value of pi
print("The value of pi is", math.pi)
```

Python import with Renaming

In Python, we can also import a module by renaming it. For example,

```
# import module by renaming it
import math as m

print(m.pi)

# Output: 3.141592653589793
```

Run Code »

Python from...import statement

We can import specific names from a module without importing the module as a whole. For example,

```
# import only pi from math module
from math import pi

print(pi)

# Output: 3.141592653589793
```

Import all names

In Python, we can import all names(definitions) from a module using the following construct:

```
# import all names from the standard module math
from math import *

print("The value of pi is", pi)
```

Run Code »

Hands on Exercise Modules

```
SimpleCalculator/
├── addition.py
├── subtraction.py
├── multiplication.py
└── division.py
```

Implement Each Module:

- `addition.py`:

```python
# addition.py
def add(a, b):
    return a + b
```

- `subtraction.py`:

```python
# subtraction.py
def subtract(a, b):
    return a - b
```

- `multiplication.py`:

```python
# multiplication.py
def multiply(a, b):
    return a * b
```

- `division.py`:

```python
# division.py
def divide(a, b):
    if b != 0:
        return a / b
    else:
        return "Cannot divide by zero!"
```

Create a Main Script (`main.py`): Now, create a `main.py` file to call these modules and perform calculations.

python Copy code

```python
# main.py
import addition
import subtraction
import multiplication
import division

def mymain():
    a = 10
    b = 5

    print(f"{a} + {b} = {addition.add(a, b)}")
    print(f"{a} - {b} = {subtraction.subtract(a, b)}")
    print(f"{a} * {b} = {multiplication.multiply(a, b)}")
    print(f"{a} / {b} = {division.divide(a, b)}")
```

```python
if __name__ == "__main__":
    mymain()
```

Question:

You have been given the task to build a Temperature Converter project in Python. The project should include the following functionalities:

1. **Celsius to Fahrenheit conversion.**
2. **Fahrenheit to Celsius conversion.**

Your task is to:

- **Create two separate modules (`celsius_to_fahrenheit.py` and `fahrenheit_to_celsius.py`), each containing a function to perform the respective conversion.**
- **Write a `main.py` file to call these functions and display the results for a given temperature (e.g., convert 100°C to Fahrenheit and 212°F to Celsius).**

Bonus Question:

- **How would you modify the program to include another module for Kelvin to Celsius conversion?**

Here's a simplified project structure for a **"temperature_converter"** app, which converts temperatures between Celsius and Fahrenheit. This project focuses on using built-in modules, custom modules, and organizing code modularly without needing an API.

Project Structure

bash Copy code

```
temperature_converter/
├── main.py              # Entry point for the project
├── utils/               # Custom modules to modularize code
│   ├── converter.py     # Module for temperature conversion functions
│   ├── display.py       # Module for displaying results
├── config.py            # Configuration file for reusable constants
└── README.md            # Project documentation
```

Files and Code

1. `main.py` - Entry Point

This script imports and uses custom modules to convert and display temperatures.

python Copy code

```python
from utils.converter import celsius_to_fahrenheit, fahrenheit_to_celsius
from utils.display import display_conversion
from datetime import datetime

def main():
    # Print current date and time using the built-in datetime module
    print(f"Temperature Converter - {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}"

    # Example conversions
    celsius_temp = 25
    fahrenheit_temp = celsius_to_fahrenheit(celsius_temp)
    display_conversion(celsius_temp, "Celsius", fahrenheit_temp, "Fahrenheit")

    fahrenheit_temp = 77
    celsius_temp = fahrenheit_to_celsius(fahrenheit_temp)
    display_conversion(fahrenheit_temp, "Fahrenheit", celsius_temp, "Celsius")

if __name__ == "__main__":
    main()
```

2. `utils/converter.py` **- Conversion Module**

Contains functions to convert temperatures between Celsius and Fahrenheit.

python Copy code

```python
# utils/converter.py

def celsius_to_fahrenheit(celsius):
    return celsius * 9/5 + 32

def fahrenheit_to_celsius(fahrenheit):
    return (fahrenheit - 32) * 5/9
```

3. `utils/display.py` **- Display Module**

Handles the display of conversion results.

python Copy code

```python
# utils/display.py

def display_conversion(from_temp, from_unit, to_temp, to_unit):
    print(f"{from_temp}° {from_unit} is equivalent to {to_temp:.2f}° {to_unit}.")
```

4. `config.py` **- Configuration File**

Contains constants used throughout the project (not strictly necessary in this example but included to show modularity).

python Copy code

```python
# config.py

DEFAULT_CELSIUS = 25  # Default Celsius temperature for demonstration
DEFAULT_FAHRENHEIT = 77  # Default Fahrenheit temperature for demonstration
```

Errors

1E: Errors

As you do more and more programming, you will naturally encounter a lot of errors (or *bugs*). Causing, understanding, and fixing errors is an important part of programming. Python will do its best to run anything that you tell it to run, but if it can't understand what you're asking, then it won't run the program. All the same, Python will try to tell you a little bit of information about what went wrong, in order to help you try to fix it.

Here are two Python errors.

Example: **A Syntax Error**

```
print "Gee golly"
```

Run program

In this first example, we forget to use the parenthesis that are required by `print(...)`. Python does not understand what you are trying to do.

Common Syntax Errors in Python

Here are a few additional examples of syntax errors that can occur in Python. One very general thing that can occur is that Python will encounter a special symbol in a place that it doesn't expect.

Example: **Syntax Error**

```
print(Hello, World!)
```

Run program

Python says `SyntaxError: invalid syntax` and points with `^` to the exclamation point. The problem is that `!` has no meaning in Python. The syntax error would go away if we had put `print("Hello, World!")` instead, because then Python would understand that the `!` is part of the text with `Hello, World`.

Here is another syntax error that is more subtle.

Example: **Syntax Error**

```
class = "Advanced Computronics for Beginners"
```

Run program

The problem is that `class` is a special word in Python. if you had written `course` instead of class it would have been fine. Click here to see the list of all special "keywords" in Python.

If you are using quotes around text and you forget the second one, or you are using parentheses and forget the second one, you will get syntax errors:

Example: **Syntax Error**

Forgetting the second quote

```
name = "Jim
```

Run program

In this error, **EOL** is short for **E**nd **O**f **L**ine: Python expected another `"` but the line ended before it was found.

Example: **Syntax Error**

Forgetting the second parenthesis

```
print(77
```

Run program

Similarly, **EOF** is short for **E**nd **O**f **F**ile: Python kept looking for a `)` but the program file ended before it was found.

Sometimes two very similar syntax errors can give two very different error messages. But, every error message is indeed trying to tell you something helpful.

Run-Time Errors

Here are a few common run-time errors. Python is able to understand what the program says, but runs into problems when actually performing the instructions.

- using an undefined variable or function. This can also occur if you use capital letters inconsistently in a variable name:

  Example

  An undefined variable

  ```
  callMe = "Maybe"
  print(callme)
  ```

  Run program

- dividing by zero, which makes no sense in mathematics. (Why? Since 0 times any number is 0, there is no solution to 1 = *X* * 0, so 1/0 is undefined.)

  Example

  Dividing by zero

  ```
  print(1/0)
  ```

  Run program

- using operators on the wrong type of data

  Example

  Adding text and a number

  ```
  print("you cannot add text and numbers" + 12)
  ```

  Run program

Logic Errors

Your program might run without crashing (no syntax or run-time errors), but still do the wrong thing. For example, perhaps you want a program to calculate the *average* of two numbers: the average of *x* and *y* is defined as

$$\frac{x+y}{2}$$

Why doesn't this program work?

Example

This does *not* calculate the average correctly.

```
x = 3
y = 4
average = x + y / 2
print(average)
```

Run program

The average should be

$$\frac{x+y}{2} = \frac{3+4}{2} = \frac{7}{2} = 3.5$$

but the program prints `5.0` instead! The error this time has to do with the "order of operations" in arithmetic. When you write `x + y / 2`, this has the same mathematical meaning as $x + (y/2) = 3 + (4/2) = 3 + 2 = 5$. To fix the problem, the third line of our program should be written as `average = (x + y) / 2`, which makes clear to Python

Exception Handling

Python try...except Block

The `try...except` block is used to handle exceptions in Python. Here's the syntax of `try...except` block:

```
try:
    # code that may cause exception
except:
    # code to run when exception occurs
```

Here, we have placed the code that might generate an exception inside the `try` block. Every `try` block is followed by an `except` block.

When an exception occurs, it is caught by the `except` block. The `except` block cannot be used without the try block.

Example: Exception Handling Using try...except

```
try:
    numerator = 10
    denominator = 0

    result = numerator/denominator

    print(result)
except:
    print("Error: Denominator cannot be 0.")

# Output: Error: Denominator cannot be 0.
```

```
try:

    even_numbers = [2,4,6,8]
    print(even_numbers[5])

except ZeroDivisionError:
    print("Denominator cannot be 0.")

except IndexError:
    print("Index Out of Bound.")

# Output: Index Out of Bound
```

Python try with else clause

In some situations, we might want to run a certain block of code if the code block inside `try` runs without any errors.

For these cases, you can use the optional `else` keyword with the `try` statement.

Let's look at an example:

```
# program to print the reciprocal of even numbers

try:
    num = int(input("Enter a number: "))
    assert num % 2 == 0
except:
    print("Not an even number!")
else:
    reciprocal = 1/num
    print(reciprocal)
```

Python try...finally

In Python, the `finally` block is always executed no matter whether there is an exception or not.

The `finally` block is optional. And, for each `try` block, there can be only one `finally` block.

Let's see an example,

```
try:
    numerator = 10
    denominator = 0

    result = numerator/denominator

    print(result)
except:
    print("Error: Denominator cannot be 0.")

finally:
    print("This is finally block.")
```

PDB()

Here's a simple project structure for a **"calculator"** app that performs basic arithmetic operations (addition, subtraction, multiplication, and division). This example demonstrates using modules, exception handling, and a built-in module.

Project Structure

bash Copy code

```
calculator/
├── main.py              # Entry point for the project
├── utils/               # Custom modules to modularize code
│   ├── operations.py    # Module for arithmetic operations
│   ├── display.py       # Module for displaying results
├── config.py            # Configuration file for default values
└── README.md            # Project documentation
```

Files and Code

1. `main.py` - Entry Point

This script handles user input, performs calculations using custom modules, and includes exception handling to manage invalid inputs (like dividing by zero).

python Copy code

```python
from utils.operations import add, subtract, multiply, divide
from utils.display import display_result
from datetime import datetime

def main():
    print(f"Calculator App - {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")

    try:
        # Sample numbers for operations
        num1 = 10
        num2 = 0

        # Perform calculations and display results
        display_result(num1, num2, add(num1, num2), "addition")
        display_result(num1, num2, subtract(num1, num2), "subtraction")
        display_result(num1, num2, multiply(num1, num2), "multiplication")

        # Handling division by zero in the divide function
        result = divide(num1, num2)
        display_result(num1, num2, result, "division")

    except ZeroDivisionError as e:
        print(f"Error: {e}")

if __name__ == "__main__":
    main()
```

2. `utils/operations.py` - Operations Module

Contains functions to perform basic arithmetic operations, with exception handling for division by zero.

python Copy code

```python
# utils/operations.py

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        raise ZeroDivisionError("Cannot divide by zero!")
    return a / b
```

3. `utils/display.py` - Display Module

Displays the results of calculations.

python Copy code

```python
# utils/display.py

def display_result(num1, num2, result, operation):
    print(f"The result of {operation} between {num1} and {num2} is: {result}")
```

4. `config.py` - Configuration File

Stores default values or other configurations (included to show modularity).

python Copy code

```python
# config.py

DEFAULT_NUM1 = 10
DEFAULT_NUM2 = 0  # Intentionally zero to trigger exception handling
```

OOP concept

Python Class and Object

```
class Parrot:

    # class attribute
    name = ""
    age = 0

# create parrot1 object
parrot1 = Parrot()
parrot1.name = "Blu"
parrot1.age = 10

# create another object parrot2
parrot2 = Parrot()
parrot2.name = "Woo"
parrot2.age = 15

# access attributes
print(f"{parrot1.name} is {parrot1.age} years old")
print(f"{parrot2.name} is {parrot2.age} years old")
```

Python Inheritance

Inheritance is a way of creating a new class for using details of an existing class without modifying it.

The newly formed class is a derived class (or child class). Similarly, the existing class is a base class (or parent class).

Example 2: Use of Inheritance in Python

Example 2: Use of Inheritance in Python

```
# base class
class Animal:

    def eat(self):
        print( "I can eat!")

    def sleep(self):
        print("I can sleep!")

# derived class
class Dog(Animal):

    def bark(self):
        print("I can bark! Woof woof!!")

# Create object of the Dog class
dog1 = Dog()

# Calling members of the base class
dog1.eat()
dog1.sleep()

# Calling member of the derived class
dog1.bark();
```

Python Encapsulation

Encapsulation is one of the key features of object-oriented programming. Encapsulation refers to the bundling of attributes and methods inside a single class.

It prevents outer classes from accessing and changing attributes and methods of a class. This also helps to achieve **data hiding**.

In Python, we denote private attributes using underscore as the prefix i.e single `_` or double `__`. For example,

```
class Computer:

    def __init__(self):
        self.__maxprice = 900

    def sell(self):
        print("Selling Price: {}".format(self.__maxprice))

    def setMaxPrice(self, price):
        self.__maxprice = price

c = Computer()
c.sell()

# change the price
c.__maxprice = 1000
c.sell()

# using setter function
c.setMaxPrice(1000)
c.sell()
```

Polymorphism

Polymorphism is another important concept of object-oriented programming. It simply means more than one form.

That is, the same entity (method or operator or object) can perform different operations in different scenarios.

In the above example, we have created a superclass: `Polygon` and two subclasses: `Square` and `Circle`. Notice the use of the `render()` method.

The main purpose of the `render()` method is to render the shape. However, the process of rendering a square is different from the process of rendering a circle.

Hence, the `render()` method behaves differently in different classes. Or, we can say `render()` is polymorphic.

To learn more about polymorphism, visit Polymorphism in Python.

```
class Polygon:
    # method to render a shape
    def render(self):
        print("Rendering Polygon...")

class Square(Polygon):
    # renders Square
    def render(self):
        print("Rendering Square...")

class Circle(Polygon):
    # renders circle
    def render(self):
        print("Rendering Circle...")

# create an object of Square
s1 = Square()
s1.render()

# create an object of Circle
c1 = Circle()
c1.render()
```

Example: Python Multiple Inheritance

```
class Mammal:
    def mammal_info(self):
        print("Mammals can give direct birth.")

class WingedAnimal:
    def winged_animal_info(self):
        print("Winged animals can flap.")

class Bat(Mammal, WingedAnimal):
    pass

# create an object of Bat class
b1 = Bat()

b1.mammal_info()
b1.winged_animal_info()
```

Example: Python Multilevel Inheritance

```
class SuperClass:

    def super_method(self):
        print("Super Class method called")

# define class that derive from SuperClass
class DerivedClass1(SuperClass):
    def derived1_method(self):
        print("Derived class 1 method called")

# define class that derive from DerivedClass1
class DerivedClass2(DerivedClass1):

    def derived2_method(self):
        print("Derived class 2 method called")

# create an object of DerivedClass2
d2 = DerivedClass2()

d2.super_method()  # Output: "Super Class method called"

d2.derived1_method()  # Output: "Derived class 1 method called"

d2.derived2_method()  # Output: "Derived class 2 method called"
```

Self Can Be Avoided

By now you are clear that the object (instance) itself is passed along as the first argument, automatically. This implicit behavior can be avoided while making a **static** method. Consider the following simple example:

```
class A(object):

    @staticmethod
    def stat_meth():
        print("Look no self was passed")
```


Here, `@staticmethod` is a function decorator that makes `stat_meth()` static. Let us instantiate this class and call the method.

Numpy Basic Arithmetic Operation

List of Arithmetic Operations

Here's a list of various arithmetic operations along with their associated operators and built-in functions:

Element-wise Operation	Operator	Function
Addition	`+`	`add()`
Subtraction	`-`	`subtract()`
Multiplication	`*`	`multiply()`
Division	`/`	`divide()`
Exponentiation	`**`	`power()`
Modulus	`%`	`mod()`

To perform each operation, we can either use the associated operator or built-in functions. For example, **to perform addition**, we can either use the `+` operator or the `add()` built-in function.

Next, we will see examples of each of these operations.

NumPy Array Element-Wise Addition

As mentioned earlier, we can use the both `+` operator and the built-in function `add()` to perform element-wise addition between two NumPy arrays. For example,

```
import numpy as np

first_array = np.array([1, 3, 5, 7])
second_array = np.array([2, 4, 6, 8])

# using the + operator
result1 = first_array + second_array
print("Using the + operator:",result1)

# using the add() function
result2 = np.add(first_array, second_array)
print("Using the add() function:",result2)
```

Run Code »

Output

```
Using the + operator: [ 3  7 11 15]
Using the add() function: [ 3  7 11 15]
```

In the above example, first we created two arrays named: `first_array` and `second_array`.

Then, we used the `+` operator and the `add()` function to perform element-wise addition respectively.

As we can see, both `+` and `add()` give the same result.

NumPy Array Element-Wise Subtraction

In NumPy, we can either use the `-` operator or the `subtract()` function to perform element-wise subtraction between two NumPy arrays. For example,

```
import numpy as np

first_array = np.array([3, 9, 27, 81])
second_array = np.array([2, 4, 8, 16])

# using the - operator
result1 = first_array - second_array
print("Using the - operator:",result1)

# using the subtract() function
result2 = np.subtract(first_array, second_array)
print("Using the subtract() function:",result2)
```

Run Code »

Output

```
Using the - operator: [ 1  5 19 65]
Using the subtract() function: [ 1  5 19 65]
```

NumPy Array Element-Wise Multiplication

For element-wise multiplication, we can use the `*` operator or the `multiply()` function. For example,

```
import numpy as np

first_array = np.array([1, 3, 5, 7])
second_array = np.array([2, 4, 6, 8])

# using the * operator
result1 = first_array * second_array
print("Using the * operator:",result1)

# using the multiply() function
result2 = np.multiply(first_array, second_array)
print("Using the multiply() function:",result2)
```

Run Code »

Output

```
Using the * operator: [ 2 12 30 56]
Using the multiply() function: [ 2 12 30 56]
```

Here, we have used `*` and `multiply()` to demonstrate two ways of multiplying the two arrays element-wise.

NumPy Array Element-Wise Division

We can use either the `/` operator or the `divide()` function to perform element-wise division between two numpy arrays. For example,

```
import numpy as np

first_array = np.array([1, 2, 3])
second_array = np.array([4, 5, 6])

# using the / operator
result1 = first_array / second_array
print("Using the / operator:",result1)

# using the divide() function
result2 = np.divide(first_array, second_array)
print("Using the divide() function:",result2)
```

Run Code »

Output

```
Using the / operator: [0.25 0.4  0.5 ]
Using the divide() function: [0.25 0.4  0.5 ]
```

Here, we can see both `/` and `divide()` give the same result.

NumPy Array Element-Wise Exponentiation

Array exponentiation refers to raising each element of an array to a given power.

In NumPy, we can use either the `**` operator or the `power()` function to perform the element-wise exponentiation operation. For example,

```
import numpy as np

array1 = np.array([1, 2, 3])

# using the ** operator
result1 = array1 ** 2
print("Using the ** operator:",result1)

# using the power() function
result2 = np.power(array1, 2)
print("Using the power() function:",result2)
```

Run Code »

Output

```
Using the ** operator: [1 4 9]
Using the power() function: [1 4 9]
```

In the above example, we have used the `**` operator and the `power()` function to perform exponentiation operation respectively.

Here, `**` and `power()` both raise each element of `array1` to the power of **2**.

NumPy Array Element-Wise Modulus

We can perform a modulus operation in NumPy arrays using the `%` operator or the `mod()` function.

This operation calculates the remainder of element-wise division between two arrays.

Let's see an example.

```
import numpy as np

first_array = np.array([9, 10, 20])
second_array = np.array([2, 5, 7])

# using the % operator
result1 = first_array % second_array
print("Using the % operator:",result1)

# using the mod() function
result2 = np.mod(first_array, second_array)
print("Using the mod() function:",result2)
```

Run Code »

Output

```
Using the % operator: [1 0 6]
Using the mod() function: [1 0 6]
```

Here, we have performed the element-wise modulus operation using both the `%` operator

NumPy Array Functions

NumPy array functions are the built-in functions provided by NumPy that allow us to create and manipulate arrays, and perform different operations on them.

We will discuss some of the most commonly used NumPy array functions.

Common NumPy Array Functions

There are many NumPy array functions available but here are some of the most commonly used ones.

Array Operations	Functions
Array Creation Functions	`np.array()`, `np.zeros()`, `np.ones()`, `np.empty()`, etc.
Array Manipulation Functions	`np.reshape()`, `np.transpose()`, etc.
Array Mathematical Functions	`np.add()`, `np.subtract()`, `np.sqrt()`, `np.power()`, etc.
Array Statistical Functions	`np.median()`, `np.mean()`, `np.std()`, and `np.var()`.
Array Input and Output Functions	`np.save()`, `np.load()`, `np.loadtxt()`, etc.

NumPy Array Creation Functions

Array creation functions allow us to create new NumPy arrays. For example,

```
import numpy as np

# create an array using np.array()
array1 = np.array([1, 3, 5])
print("np.array():\n", array1)

# create an array filled with zeros using np.zeros()
array2 = np.zeros((3, 3))
print("\nnp.zeros():\n", array2)

# create an array filled with ones using np.ones()
array3 = np.ones((2, 4))
print("\nnp.ones():\n", array3)
```

Run Code »

Output

```
np.array():
[1 3 5]

np.zeros():
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]

np.ones():
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]]
```

NumPy Array Manipulation Functions

NumPy array manipulation functions allow us to modify or rearrange NumPy arrays. For example,

```
import numpy as np

# create a 1D array
array1 = np.array([1, 3, 5, 7, 9, 11])

# reshape the 1D array into a 2D array
array2 = np.reshape(array1, (2, 3))

# transpose the 2D array
array3 = np.transpose(array2)

print("Original array:\n", array1)
print("\nReshaped array:\n", array2)
print("\nTransposed array:\n", array3)
```

Run Code »

Output

```
Original array:
[ 1  3  5  7  9 11]

Reshaped array:
 [[ 1  3  5]
 [ 7  9 11]]

Transposed array:
 [[ 1  7]
 [ 3  9]
 [ 5 11]]
```

In this example,

- `np.reshape(array1, (2, 3))` - reshapes `array1` into 2D array with shape `(2,3)`
- `np.transpose(array2)` - transposes 2D array `array2`

NumPy Array Mathematical Functions

In NumPy, there are tons of mathematical functions to perform on arrays. For example,

```
import numpy as np

# create two arrays
array1 = np.array([1, 2, 3, 4, 5])
array2 = np.array([4, 9, 16, 25, 36])

# add the two arrays element-wise
arr_sum = np.add(array1, array2)

# subtract the array2 from array1 element-wise
arr_diff = np.subtract(array1, array2)

# compute square root of array2 element-wise
arr_sqrt = np.sqrt(array2)


print("\nSum of arrays:\n", arr_sum)
print("\nDifference of arrays:\n", arr_diff)
print("\nSquare root of first array:\n", arr_sqrt)
```

Run Code »

Output

```
Sum of arrays:
[ 5 11 19 29 41]

Difference of arrays:
 [ -3  -7 -13 -21 -31]

Square root of first array:
 [2. 3. 4. 5. 6.]
```

NumPy Array Statistical Functions

NumPy provides us with various statistical functions to perform statistical data analysis.

These statistical functions are useful to find basic statistical concepts like mean, median, variance, etc. It is also used to find the maximum or the minimum element in an array.

Let's see an example.

```
import numpy as np

# create a numpy array
marks = np.array([76, 78, 81, 66, 85])

# compute the mean of marks
mean_marks = np.mean(marks)
print("Mean:",mean_marks)

# compute the median of marks
median_marks = np.median(marks)
print("Median:",median_marks)

# find the minimum and maximum marks
min_marks = np.min(marks)
print("Minimum marks:", min_marks)

max_marks = np.max(marks)
print("Maximum marks:", max_marks)
```

Run Code »

Output

```
Mean: 77.2
Median: 78.0
Minimum marks: 66
Maximum marks: 85
```

NumPy Array Input/Output Functions

NumPy offers several input/output (I/O) functions for loading and saving data to and from files. For example,

```
import numpy as np

# create an array
array1 = np.array([[1, 3, 5], [2, 4, 6]])

# save the array to a text file
np.savetxt('data.txt', array1)

# load the data from the text file
loaded_data = np.loadtxt('array1.txt')

# print the loaded data
print(loaded_data)
```

Run Code »

Output

```
[[1. 3. 5.]
 [2. 4. 6.]]
```

In this example, we first created the 2D array named `array1` and then saved it to a text file using the `np.savetxt()` function.

We then loaded the saved data using the `np.loadtxt()` function.

Numpy Comparison/Logical Operations

NumPy provides several comparison and logical operations that can be performed on NumPy arrays.

NumPy's comparison operators allow for element-wise comparison of two arrays.

Similarly, logical operators perform boolean algebra, which is a branch of algebra that deals with `True` and `False` statements.

First we'll discuss comparison operations and then about logical operations in NumPy.

NumPy Comparison Operators

NumPy provides various element-wise comparison operators that can compare the elements of two NumPy arrays.

Here's a list of various comparison operators available in NumPy.

Here's a list of various comparison operators available in NumPy.

Operators	Descriptions
`<` (less than)	returns `True` if element of the first array is less than the second one
`<=` (less than or equal to)	returns `True` if element of the first array is less than or equal to the second one
`>` (greater than)	returns `True` if element of the first array is greater than the second one
`>=` (greater than or equal to)	returns `True` if element of the first array is greater than or equal to the second one
`==` (equal to)	returns `True` if the element of the first array is equal to the second one
`!=` (not equal to)	returns `True` if the element of the first array is not equal to the second one

Next, we'll see examples of these operators.

Example 1: NumPy Comparison Operators

```
import numpy as np

array1 = np.array([1, 2, 3])
array2 = np.array([3, 2, 1])

# less than operator
result1 = array1 < array2
print("array1 < array2:",result1)    # Output: [ True False False]

# greater than operator
result2 = array1 > array2
print("array1 > array2:",result2)    # Output: [False False  True]

# equal to operator
result3 = array1 == array2
print("array1 == array2:",result3)    # Output: [False  True False]
```

Run Code »

Output

```
array1 < array2: [ True False False]
array1 > array2: [False False  True]
array1 == array2: [False  True False]
```


Here, we can see that the output of the comparison operators is also an array, where each element is either `True` or `False` based on the array element's comparison.

NumPy Comparison Functions

NumPy also provides built-in functions to perform all the comparison operations.

For example, the `less()` function returns `True` if **each element** of the first array is less than the corresponding element in the second array.

Here's a list of all built-in comparison functions.

Functions	Descriptions
`less()`	returns element-wise `True` if the first value is less than the second
`less_equal()`	returns element-wise `True` if the first value is less than or equal to second
`greater()`	returns element-wise `True` if the first value is greater then second
`greater_equal()`	returns element-wise `True` if the first value is greater than or equal to second
`equal()`	returns element-wise `True` if two values are equal
`not_equal()`	returns element-wise `True` if two values are not equal

Next, we will see an example of all these functions.

Example 2: NumPy Comparison Functions

```
import numpy as np

array1 = np.array([9, 12, 21])
array2 = np.array([21, 12, 9])

# use of less()
result = np.less(array1, array2)
print("Using less():",result)    # Output: [ True False False]

# use of less_equal()
result = np.less_equal(array1, array2)
print("Using less_equal():",result)     # Output: [ True  True False]

# use of greater()
result = np.greater(array1, array2)
print("Using greater():",result)    # Output: [False False  True]

# use of greater_equal()
result = np.greater_equal(array1, array2)
print("Using greater_equal():",result)    # Output: [False  True  True]

# use of equal()
result = np.equal(array1, array2)
print("Using equal():",result)    # Output: [False  True False]

# use of not_equal()
result = np.not_equal(array1, array2)
print("Using not_equal():",result)    # Output: [ True False  True]
```

Run Code »

Output

```
Using less(): [ True False False]
Using less_equal(): [ True  True False]
Using greater(): [False False  True]
Using greater_equal(): [False  True  True]
Using equal(): [False  True False]
Using not_equal(): [ True False  True]
```

NumPy Logical Operations

As mentioned earlier, logical operators perform Boolean algebra; a branch of algebra that deals with `True` and `False` statements.

Logical operations are performed element-wise. For example, if we have two arrays `x1` and `x2` of the same shape, the output of the logical operator will also be an array of the same shape.

Here's a list of various logical operators available in NumPy:

Operators	Descriptions
`logical_and`	Computes the element-wise truth value of **x1** `AND` **x2**
`logical_or`	Computes the element-wise truth value of **x1** `OR` **x2**
`logical_not`	Computes the element-wise truth value of `NOT` **x**

Next, we will see examples of these operators.

Example 3: NumPy Logical Operations

```
import numpy as np

x1 = np.array([True, False, True])
x2 = np.array([False, False, True])

# Logical AND
print(np.logical_and(x1, x2)) # Output: [False False  True]

# Logical OR
print(np.logical_or(x1, x2)) # Output: [ True False  True]

# Logical NOT
print(np.logical_not(x1)) # Output: [False  True False]
```

Run Code »

Here, we have performed logical operations on two arrays, `x1` and `x2`, containing boolean values.

It demonstrates the logical **AND**, **OR**, and **NOT** operations using `np.logical_and()`, `np.logical_or()`, and `np.logical_not()` respectively.

NumPy Math Functions

Numpy provides a wide range of mathematical functions that can be performed on arrays.

Let's explore three different types of math functions in NumPy:

1. Trigonometric Functions
2. Arithmetic Functions
3. Rounding Functions

1. Trigonometric Functions

NumPy provides a set of standard trigonometric functions to calculate the trigonometric ratios (sine, cosine, tangent, etc.)

Here's a list of commonly used trigonometric functions in NumPy.

Trigonometric Function	Computes (in radians)
`sin()`	the sine of an angle
`cos()`	cosine of an angle
`tan()`	tangent of an angle
`arcsin()`	the inverse sine
`arccos()`	the inverse cosine
`arctan()`	the inverse tangent
`degrees()`	converts an angle in radians to degrees
`radians()`	converts an angle in degrees to radians

Let's see the examples.

Let's see the examples.

```
import numpy as np

# array of angles in radians
angles = np.array([0, 1, 2])
print("Angles:", angles)

# compute the sine of the angles
sine_values = np.sin(angles)
print("Sine values:", sine_values)

# compute the inverse sine of the angles
inverse_sine = np.arcsin(angles)
print("Inverse Sine values:", inverse_sine)
```

Run Code »

Output

```
Angles: [0 1 2]
Sine values: [0.         0.84147098 0.90929743]
Inverse Sine values: [0.         1.57079633        nan]
```

In this example, the `sin()` and `arcsin()` functions calculate the sine and inverse sine values, respectively, for each element in the `angles` array.

The resulting values are in radians.

Now let's see the examples for `degrees()` and `radians()`.

Now let's see the examples for `degrees()` and `radians()`.

```
import numpy as np

# define an angle in radians
angle =  1.57079633
print("Initial angle in radian:", angle)

# convert the angle to degrees
angle_degree = np.degrees(angle)
print("Angle in degrees:", angle_degree)

# convert the angle back to radians
angle_radian = np.radians(angle_degree)
print("Angle in radians (after conversion):", angle_radian)
```

Run Code »

Output

```
Initial angle in radian: 1.57079633
Angle in degrees: 90.0000001836389
Angle in radians (after conversion): 1.57079633
```

Here, we first initialized an angle in radians. Then we converted it to degrees using the `degrees()` function.

Numpy Constants

In Numpy, the most commonly used constants are `pi` and `e`.

np.pi

`np.pi` is a mathematical constant that returns the value of pi(π) as a floating point number. Its value is approximately **3.141592653589793**.

Let's see an example.

```
import numpy as np

radius = 2
circumference = 2 * np.pi * radius
print(circumference)
```

Run Code »

Output

```
12.566370614359172
```

Here, the `np.pi` returns the value **3.141592653589793**, which calculates the circumference.

Instead of the long floating point number, we used the constant `np.pi`. This made our code look cleaner.

np.e

`np.e` is widely used with exponential and logarithmic functions. Its value is approximately **2.718281828459045**.

Let's see an example.

```
import numpy as np

y = np.e

print(y)
```

Run Code »

Output

```
2.718281828459045
```

In the above example, we simply printed the constant `np.e`. As we can see, it returns its value **2.718281828459045**.

However, this example makes no sense because that's not how we use the constant `e` in real life.

We usually use the constant `e` with the function `exp()`. `e` is the base of exponential function, `exp(x)`, which is equivalent to `e^x`.

We usually use the constant `e` with the function `exp()`. `e` is the base of exponential function, `exp(x)`, which is equivalent to `e^x`.

Let's see an example.

```
import numpy as np

x = 1
y = np.exp(x)

print(y)
```

Run Code »

Output

```
2.718281828459045
```

As you can see, we got the same output like in the previous example. This is because

```
np.exp(x)
```

is equivalent to

```
np.e ^ x
```

The value of `x` is **1**. So, `np.e^x` returns the value of the constant `e`.

Arithmetic Operations with Numpy Constants and Arrays

We can use arithmetic operators to perform operations such as addition, subtraction, and division between a constant and all elements in an array.

Let's see an example.

```
import numpy as np

array1 = np.array([1, 2, 3])

# add each array element with the constant pi
y = array1 + np.pi

print(y)
```

Run Code »

Output

```
[4.14159265 5.14159265 6.14159265]
```

In this example, the value **3.141592653589793** (π) is added with each element of `array1`.

NumPy Statistical Functions

Statistics involves gathering data, analyzing it, and drawing conclusions based on the information collected.

NumPy provides us with various statistical functions that can perform statistical data analysis.

Common NumPy Statistical Functions

Here are some of the statistical functions provided by NumPy:

Functions	Descriptions
`median()`	return the median of an array
`mean()`	return the mean of an array
`std()`	return the standard deviation of an array
`percentile()`	return the nth percentile of elements in an array
`min()`	return the minimum element of an array
`max()`	return the maximum element of an array

Next, we will see examples using these functions.

Find Median Using NumPy

The median value of a numpy array is the middle value in a **sorted array**.

In other words, it is the value that separates the higher half from the lower half of the data.

Suppose we have the following list of numbers:

```
1, 5, 7, 8, 9, 12, 14
```

Then, median is simply the middle number, which in this case is **8**.

It is important to note that if the number of elements is

- **Odd**, the median is the middle element.
- **Even**, the median is the average of the two middle elements.

Now, we will learn how to calculate the median using NumPy for arrays with odd and even number of elements.

Example 1: Compute Median for Odd Number of Elements

```
import numpy as np

# create a 1D array with 5 elements
array1 = np.array([1, 2, 3, 4, 5])

# calculate the median
median = np.median(array1)

print(median)

# Output: 3.0
```

Run Code »

In the above example, the array named `array1` contains an odd number of elements (**5** elements).

So, `np.median(array1)` returns the median of `array1` as **3**, which is the middle value of the sorted array.

Example 2: Compute Median for Even Number of Elements

```
import numpy as np

# create a 1D array with 6 elements
array1 = np.array([1, 2, 3, 4, 5, 7])

# calculate the median
median = np.median(array1)
print(median)

# Output: 3.5
```

Run Code »

Here, since the `array1` array has an even number of elements (**6** elements), the median is calculated as the average of the two middle elements (**3** and **4**) i.e. **3.5.**

Median of NumPy 2D Array

Calculation of the median is not just limited to 1D array. We can also calculate the median of the 2D array.

In a 2D array, median can be calculated either along the horizontal or the vertical axis individually, or across the entire array.

When computing the median of a 2D array, we use the `axis` parameter inside `np.median()` to specify the axis along which to compute the median.

If we specify,

- `axis = 0`, median is calculated along vertical axis
- `axis = 1`, median is calculated along horizontal axis

If we don't use the `axis` parameter, the median is computed over the entire array.

Example: Compute the median of a 2D array

```
import numpy as np

# create a 2D array
array1 = np.array([[2, 4, 6],
                   [8, 10, 12],
                   [14, 16, 18]])

# compute median along horizontal axis
result1 = np.median(array1, axis=1)

print("Median along horizontal axis :", result1)

# compute median along vertical axis
result2 = np.median(array1, axis=0)

print("Median along vertical axis:", result2)

# compute median of entire array
result3 = np.median(array1)

print("Median of entire array:", result3)
```

Run Code »

Output

```
Median along horizontal axis : [ 4. 10. 16.]
Median along vertical axis: [ 8. 10. 12.]
Median of entire array: 10.0
```

In this example, we have created a 2D array named `array1`.

We then computed the median along the horizontal and vertical axis individually and then computed the median of the entire array.

We then computed the median along the horizontal and vertical axis individually and then computed the median of the entire array.

- `np.median(array1, axis=1)` - median along horizontal axis, which gives `[4. 10. 16.]`
- `np.median(array1, axis=0)` - median along vertical axis, which gives `[8. 10. 12.]`
- `np.median(array1)` - median over the entire array, which gives `10.0`

Compute Mean Using NumPy

The mean value of a NumPy array is the average value of all the elements in the array.

It is calculated by adding all elements in the array and then dividing the result by the total number of elements in the array.

We use the `np.mean()` function to calculate the mean value. For example,

```
import numpy as np

# create a numpy array
marks = np.array([76, 78, 81, 66, 85])

# compute the mean of marks
mean_marks = np.mean(marks)

print(mean_marks)

# Output: 77.2
```

Run Code »

In this example, the mean value is **77.2**, which is calculated by adding the elements (**76, 78, 81, 66, 85**) and dividing the result by **5** (total number of array elements).

Example 3: Mean of NumPy N-d Array

```
import numpy as np

# create a 2D array
array1 = np.array([[1, 3],
                   [5, 7]])

# calculate the mean of the entire array
result1 = np.mean(array1)
print("Entire Array:",result1)  # 4.0

# calculate the mean along vertical axis (axis=0)
result2 = np.mean(array1, axis=0)
print("Along Vertical Axis:",result2)  # [3. 5.]

# calculate the mean along  (axis=1)
result3 = np.mean(array1, axis=1)
print("Along Horizontal Axis :",result3)  # [2. 6.]
```

Run Code »

Output

```
Entire Array: 4.0
Along Vertical Axis: [3. 5.]
Along Horizontal Axis : [2. 6.]
```

Here, first we have created the 2D array named `array1`. We then calculated the mean using `np.mean()`.

- `np.mean(array1)` - calculates the mean over the entire array
- `np.mean(array1, axis=0)` - calculates the mean along vertical axis
- `np.mean(array1, axis=1)` calculates the mean along horizontal axis

Standard Deviation of NumPy Array

The standard deviation is a measure of the spread of the data in the array. It gives us the degree to which the data points in an array deviate from the mean.

- Smaller standard deviation indicates that the data points are closer to the mean
- Larger standard deviation indicates that the data points are more spread out.

In NumPy, we use the `np.std()` function to calculate the standard deviation of an array.

Example: Compute the Standard Deviation in NumPy

```
import numpy as np

# create a numpy array
marks = np.array([76, 78, 81, 66, 85])

# compute the standard deviation of marks
std_marks = np.std(marks)
print(std_marks)

# Output: 6.803568381206575
```

Run Code »

In the above example, we have used the `np.std()` function to calculate the standard deviation of the `marks` array.

Here, `6.803568381206575` is the standard deviation of `marks`. It tells us how much the values in the `marks` array deviate from the mean value of the array.

Standard Deviation of NumPy 2D Array

In a 2D array, standard deviation can be calculated either along the horizontal or the vertical axis individually, or across the entire array.

Similar to mean and median, when computing the standard deviation of a 2D array, we use the `axis` parameter inside `np.std()` to specify the axis along which to compute the standard deviation.

Example: Compute the Standard Deviation of a 2D array.

```
import numpy as np

# create a 2D array
array1 = np.array([[2, 5, 9],
                   [3, 8, 11],
                   [4, 6, 7]])

# compute standard deviation along horizontal axis
result1 = np.std(array1, axis=1)
print("Standard deviation along horizontal axis:", result1)

# compute standard deviation along vertical axis
result2 = np.std(array1, axis=0)
print("Standard deviation  along vertical axis:", result2)

# compute standard deviation of entire array
result3 = np.std(array1)
print("Standard deviation of entire array:", result3)
```

Run Code »

Output

```
Standard deviation along horizontal axis: [2.86744176 3.29983165 1.24721913]
Standard deviation along vertical axis: [0.81649658 1.24721913 1.63299316]
Standard deviation of entire array: 2.7666443551086073
```

Here, we have created a 2D array named `array1`.

We then computed the standard deviation along horizontal and vertical axis individually and then computed the standard deviation of the entire array.

Compute Percentile of NumPy Array

In NumPy, we use the `percentile()` function to compute the nth percentile of a given array.

Let's see an example.

```
import numpy as np

# create an array
array1 = np.array([1, 3, 5, 7, 9, 11, 13, 15, 17, 19])

# compute the 25th percentile of the array
result1 = np.percentile(array1, 25)
print("25th percentile:",result1)

# compute the 75th percentile of the array
result2 = np.percentile(array1, 75)
print("75th percentile:",result2)
```

Run Code »

Output

```
25th percentile: 5.5
75th percentile: 14.5
```

Here,

- **25%** of the values in `array1` are less than or equal to **5.5**.
- **75%** of the values in `array1` are less than or equal to **14.5**.

> **Note:** To learn more about percentile, visit *NumPy Percentile*.

Find Minimum and Maximum Value of NumPy Array

We use the `min()` and `max()` function in NumPy to find the minimum and maximum values in a given array.

Let's see an example.

```
import numpy as np

# create an array
array1 = np.array([2,6,9,15,17,22,65,1,62])

# find the minimum value of the array
min_val = np.min(array1)

# find the maximum value of the array
max_val = np.max(array1)

# print the results
print("Minimum value:", min_val)
print("Maximum value:", max_val)
```

Run Code »

Output

```
Minimum value: 1
Maximum value: 65
```

As we can see `min()` and `max()` returns the minimum and maximum value of `array1` which is **1** and **65** respectively.

> **Note:** To learn more about `min()` and `max()`, visit *NumPy min()* and *NumPy max()*.

NumPy String Functions

In addition to NumPy's numerical capabilities, it also provides several functions that can be applied to strings represented in NumPy arrays.

For example, we can add two strings, change the contents of a string, case conversion, padding, trimming, and so on.

Common NumPy String Functions

Here are some of the string functions provided by NumPy:

Functions	Descriptions
`add()`	concatenates two strings
`multiply()`	repeats a string for a specified number of times
`capitalize()`	capitalizes the first letter of a string
`lower()`	converts all uppercase characters in a string to lowercase
`upper()`	converts all lowercase characters in a string to uppercase
`join()`	joins a sequence of strings
`equal()`	checks if two strings are equal or not

NumPy String add() Function

In NumPy, we use the `np.char.add()` function to perform element-wise string concatenation for two arrays of strings. For example,

```
import numpy as np

array1 = np.array(['iPhone: ', 'price: '])
array2 = np.array(['15', '$900'])

# perform element-wise array string concatenation
result = np.char.add(array1, array2)

print(result)
```

Run Code »

Output

```
['iPhone: 15' 'price: $900']
```

In this example, we have defined two NumPy arrays: `array1` and `array2`, with string elements `'iphone: '` and `'price: '` and `'15'` and `'$900'` respectively.

We then used `np.char.add(array1, array2)` to concatenate the elements of `array1` and `array2` element-wise.

Finally, we have stored the concatenated string in `result`, which in our case are `'iPhone: 15'` and `'price: $900'`.

NumPy String multiply() Function

The `np.char.multiply()` function is used to perform element-wise string repetition. It returns an array of strings with each string element repeated a specified number of times.

Let's see an example.

```
import numpy as np

# define array with three string elements
array1 = np.array(['A', 'B', 'C'])

#  repeat each element in array1 two times
result = np.char.multiply(array1, 2)

print(result)

# Output: ['AA' 'BB' 'CC']
```

Run Code »

Here, `np.char.multiply(array1, 2)` repeats each element in `array1` two times.

NumPy String capitalize() Function

In NumPy, the `np.char.capitalize()` function is used to capitalize the first character of each string element in a given array. For example,

```
import numpy as np

# define an array with three string elements
array1 = np.array(['eric', 'paul', 'sean'])

# capitalize the first letter of each string in array1
result = np.char.capitalize(array1)

print(result)

# Output: ['Eric' 'Paul' 'Sean']
```

Run Code »

In this example, `np.char.capitalize(array1)` capitalizes the first character of the string element in the `array1` array.

NumPy String upper() and lower() Function

NumPy provides two string functions for converting the case of a string: `np.char.upper()` and `np.char.lower()`.

Let's see an example.

```
import numpy as np

array1 = np.array(['nEpalI', 'AmeriCAN', 'CaNadIan'])

# convert all string elements to uppercase
result1 = np.char.upper(array1)

# convert all string elements to lowercase
result2 = np.char.lower(array1)

print("To Uppercase: ",result1)
print("To Lowercase: ",result2)
```

Run Code »

Output

```
To Uppercase:  ['NEPALI' 'AMERICAN' 'CANADIAN']
To Lowercase:  ['nepali' 'american' 'canadian']
```

Here,

- `np.char.upper(array1)` - convert all strings in `array1` to uppercase
- `np.char.lower(array1)` - convert all strings in `array1` to lowercase

NumPy String join() Function

In NumPy, we use the `np.char.join()` function to join strings in an array with a specified delimiter. For example,

```
import numpy as np

# create an array of strings
array1 = np.array(['hello', 'world'])

# join the strings in the array using a dash as the delimiter
result = np.char.join('-', array1)

print(result)

# Output: ['h-e-l-l-o' 'w-o-r-l-d']
```

Run Code »

In this example, the `np.char.join('-', array1)` function is used to join the strings in `array1` using dash `-` as the delimiter.

The resulting strings have each character separated by a dash.

NumPy String equal() Function

The `np.char.equal()` function is used to perform element-wise string comparison between two string arrays.

Let's see an example.

```
import numpy as np

# create two arrays of strings
array1 = np.array(['C', 'Python', 'Swift'])
array2 = np.array(['C++', 'Python', 'Java'])

# check if each element of the arrays is equal
result = np.char.equal(array1, array2)

print(result)

# Output: [False True False]
```

Run Code »

Here, `np.char.equal(array1, array2)` checks whether the corresponding elements of `array1` and `array2` are equal or not.

The resulting boolean array `[False True False]` indicates that the first and third elements of `array1` and `array2` are not equal, while the second element is equal.

Access Array Elements Using Index

We can use indices to access individual elements of a NumPy array.

Suppose we have a NumPy array:

```
array1 = np.array([1, 3, 5, 7, 9])
```

Now, we can use the index number to access array elements as:

- `array1[0]` - to access the first element, i.e. **1**
- `array1[2]` - to access the third element, i.e. **5**
- `array1[4]` - to access the fifth element, i.e. **9**

Example: Access Array Elements Using Index

```
import numpy as np

array1 = np.array([1, 3, 5, 7, 9])

# access numpy elements using index
print(array1[0])    # prints 1
print(array1[2])    # prints 5
print(array1[4])    # prints 9
```

Run Code »

Modify Array Elements Using Index

We can use indices to change the value of an element in a NumPy array. For example,

```
import numpy as np

# create a numpy array
numbers = np.array([2, 4, 6, 8, 10])

# change the value of the first element
numbers[0] = 12
print("After modifying first element:",numbers)    # prints [12 4 6 8 10]

# change the value of the third element
numbers[2] = 14
print("After modifying third element:",numbers)    # prints [12 4 14 8 10]
```

Run Code »

Output

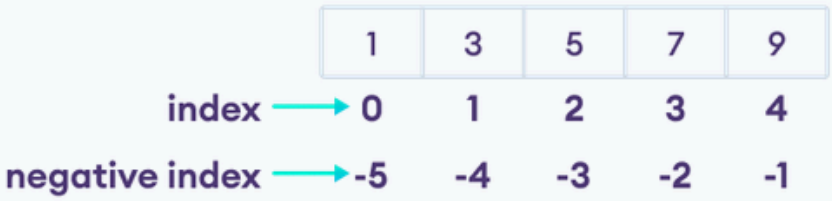
```
After modifying first element: [12  4  6  8 10]
After modifying third element: [12  4 14  8 10]
```

In the above example, we have modified elements of the `numbers` array using array indexing.

- `numbers[0] = 12` - modifies the first element of `numbers` and sets its value to **12**
- `numbers[2] = 14` - modifies the third element of `numbers` and sets its value to **14**

NumPy Negative Array Indexing

NumPy allows negative indexing for its array. The index of **-1** refers to the last item, **-2** to the second last item and so on.



NumPy Array Negative Indexing

Let's see an example.

```
import numpy as np

# create a numpy array
numbers = np.array([1, 3, 5, 7, 9])

# access the last element
print(numbers[-1])    # prints 9

# access the second-to-last element
print(numbers[-2])    # prints 7
```

Run Code »

Modify Array Elements Using Negative Indexing

Similar to regular indexing, we can also modify array elements using negative indexing. For example,

```
import numpy as np

# create a numpy array
numbers = np.array([2, 3, 5, 7, 11])

# modify the last element
numbers[-1] = 13
print(numbers)    # Output: [2 3 5 7 13]

# modify the second-to-last element
numbers[-2] = 17
print(numbers)    # Output: [2 3 5 17 13]
```

Run Code »

Here, `numbers[-1] = 13` modifies the last element to **13** and `numbers[-2] = 17` modifies the second-to-last element to **17**.

2-D NumPy Array Indexing

Array indexing in NumPy allows us to access and manipulate elements in a 2-D array.

To access an element of `array1`, we need to specify the row index and column index of the element. Suppose we have following 2-D array,

```
array1 = np.array([[1, 3, 5],
                   [7, 9, 2],
                   [4, 6, 8]])
```

Now, say we want to access the element in the third row and second column we specify the index as:

```
array1[2, 1] # returns 6
```

Since we know indexing starts from **0**. So to access the element in the third row and second column, we need to use index **2** for the third row and index **1** for the second column respectively.

Example: 2-D NumPy Array Indexing

```
import numpy as np

# create a 2D array
array1 = np.array([[1, 3, 5, 7],
                   [9, 11, 13, 15],
                   [2, 4, 6, 8]])


# access the element at the second row and fourth column
element1 = array1[1, 3]  # returns 15
print("4th Element at 2nd Row:",element1)

# access the element at the first row and second column
element2 = array1[0, 1]  # returns 3
print("2nd Element at First Row:",element2)
```

Run Code »

Output

```
4th Element at 2nd Row: 15
2nd Element at First Row: 3
```

Access Row or Column of 2D Array Using Indexing

In NumPy, we can access specific rows or columns of a 2-D array using array indexing.

Let's see an example.

```
import numpy as np

# create a 2D array
array1 = np.array([[1, 3, 5],
                   [7, 9, 2],
                   [4, 6, 8]])

# access the second row of the array
second_row = array1[1, :]
print("Second Row:", second_row)  # Output: [7 9 2]

# access the third column of the array
third_col = array1[:, 2]
print("Third Column:", third_col)  # Output: [5 2 8]
```

Run Code »

Output

```
Second Row: [7 9 2]
Third Column: [5 2 8]
```

Here,

- `array1[1, :]` - access the second row of `array1`
- `array1[:, 2]` - access the third column of `array1`

3-D NumPy Array Indexing

We learned how to access elements in a 2D array. We can also access elements in higher dimensional arrays.

To access an element of a 3D array, we use **three indices** separated by commas.

- The **first index** refers to the slice
- The **second index** refers to the row
- The **third index** refers to the column.

> **Note**: In 3D arrays, slice is a 2D array that is obtained by taking a subset of the elements in one of the dimensions.

Let's see an example.

```
import numpy as np

# create a 3D array with shape (2, 3, 4)
array1 = np.array([[[1, 2, 3, 4],
                    [5, 6, 7, 8],
                    [9, 10, 11, 12]],

                    [[13, 14, 15, 16],
                    [17, 18, 19, 20],
                    [21, 22, 23, 24]]])

# access a specific element of the array
element = array1[1, 2, 1]

# print the value of the element
print(element)

# Output: 22
```

Run Code »

Syntax of NumPy Array Slicing

Here's the syntax of array slicing in NumPy:

```
array[start:stop:step]
```

Here,

- `start` - index of the first element to be included in the slice
- `stop` - index of the last element (exclusive)
- `step` - step size between each element in the slice

> **Note:** When we slice arrays, the start index is inclusive but the stop index is exclusive.
>
> - If we omit `start`, slicing starts from the first element
> - If we omit `stop`, slicing continues up to the last element
> - If we omit `step`, default step size is 1

1D NumPy Array Slicing

In NumPy, it's possible to access the portion of an array using the slicing operator `:`. For example,

```
import numpy as np

# create a 1D array
array1 = np.array([1, 3, 5, 7, 8, 9, 2, 4, 6])

# slice array1 from index 2 to index 6 (exclusive)
print(array1[2:6])  # [5 7 8 9]

# slice array1 from index 0 to index 8 (exclusive) with a step size of 2
print(array1[0:8:2])  # [1 5 8 2]

# slice array1 from index 3 up to the last element
print(array1[3:])  # [7 8 9 2 4 6]

# items from start to end
print(array1[:])   # [1 3 5 7 8 9 2 4 6]
```

Run Code »

In the above example, we have created the array named `array1` with **9** elements.

Then, we used the slicing operator `:` to slice array elements.

- `array1[2:6]` - slices `array1` from index **2** to index **6**, not including index **6**
- `array1[0:8:2]` - slices `array1` from index **0** to index **8**, not including index **8**
- `array1[3:]` - slices `array1` from index **3** up to the last element
- `array1[:]` - returns all items from beginning to end

2D NumPy Array Slicing

A 2D NumPy array can be thought of as a matrix, where each element has two indices, row index and column index.

To slice a 2D NumPy array, we can use the same syntax as for slicing a 1D NumPy array. The only difference is that we need to specify a slice for each dimension of the array.

Syntax of 2D NumPy Array Slicing

```
array[row_start:row_stop:row_step, col_start:col_stop:col_step]
```

Here,

- `row_start`, `row_stop`, `row_step` - specifies starting index, stopping index, and step size for the rows respectively
- `col_start`, `col_stop`, `col_step` - specifies starting index, stopping index, and step size for the columns respectively

Let's understand this with an example.

```
# create a 2D array
array1 = np.array([[1, 3, 5, 7],
                   [9, 11, 13, 15]])

print(array1[:2, :2])

# Output

[[ 1  3]
 [ 9 11]]
```

Here, the `,` in `[:2, :2]` separates the rows of the array.

The first `:2` returns first 2 rows i.e., entire `array1`. This results in

```
[1  3]
```

The second `:2` returns first 2 columns from the 2 rows. This results in

```
[9 11]
```

Example: 2D NumPy Array Slicing

```
import numpy as np

# create a 2D array
array1 = np.array([[1, 3, 5, 7],
                   [9, 11, 13, 15],
                   [2, 4, 6, 8]])


# slice the array to get the first two rows and columns
subarray1 = array1[:2, :2]

# slice the array to get the last two rows and columns
subarray2 = array1[1:3, 2:4]

# print the subarrays
print("First Two Rows and Columns: \n",subarray1)
print("Last two Rows and Columns: \n",subarray2)
```

Run Code »

NumPy Vectorization Vs Python for Loop

Even though NumPy is a Python library, it inherited vectorization from C programming. As C is efficient in terms of speed and memory, NumPy vectorization is also much faster than Python.

Let's compare the time it takes to perform a vectorized operation with that of an equivalent loop-based operation.

Python for loop

```
import time

start = time.time()

array1 = [1, 2, 3, 4, 5]

for i in range(len(array1)):
    array1[i] += 10

end = time.time()

print("For loop time:", end - start)
```

Run Code »

Output

```
For loop time: 4.76837158203125e-06
```

NumPy Vectorization

```
import numpy as np
import time

start = time.time()

array1 = np.array([1, 2, 3, 4, 5 ])

result = array1 + 10

end = time.time()

print("Vectorization time:", end - start)
```

Run Code »

Output

```
Vectorization time: 1.5020370483398438e-05
```

Here, the difference in execution time between vectorization and a for loop is significant, even for simple operation.

```
import numpy as np

# array
array1 = np.array([-1, 0, 2, 3, 4])

# function that returns the square of a positive number
def find_square(x):
    if x < 0:
        return 0
    else:
        return x ** 2
```

Run Code »

Now, to apply the function `find_square()` to `array1`, we have two options: use a loop or vectorize the operation.

Since loops are complicated and slow by nature, it's efficient and convenient to use `vectorize()`.

Let's see an example.

```
import numpy as np

# array whose square we need to find
array1 = np.array([-1, 0, 2, 3, 4])

# function to find the square
def find_square(x):
    if x < 0:
        return 0
    else:
        return x ** 2

# vectorize() to vectorize the function find_square()
vectorized_function = np.vectorize(find_square)

# passing an array to a vectorized function
result = vectorized_function(array1)
```

Install Pandas

To install pandas, you need Python and PIP installed in your system. If you have Python and PIP installed already, you can install pandas by entering the following command in the terminal:

```
pip install pandas
```

If the installation completes without any errors, Pandas is now successfully installed on your system. You can start using it in your Python projects by importing the Pandas library.

Import Pandas in Python

We can import Pandas in Python using the import statement.

```
import pandas as pd
```

The code above imports the `pandas` library into our program with the alias `pd`.

After this `import` statement, we can use Pandas functions and objects by calling them with `pd`.

For example, you can use Pandas dataframe in your program using `pd.DataFrame()`.

Create a Pandas Series

There are multiple ways to create a Pandas Series, but the most common way is by using a Python list. Let's see an example of creating a Series using a list:

```
import pandas as pd

# create a list
data = [10, 20, 30, 40, 50]

# create a series from the list
my_series = pd.Series(data)

print(my_series)
```

Run Code »

Output

```
0    10
1    20
2    30
3    40
4    50
dtype: int64
```

In this example, we created a Python list called `data` containing five integer values. We then passed this list to the `Series()` function, which converted it into a Pandas Series called `my_series`.

Labels

The labels in the Pandas Series are index numbers by default. Like in dataframe and array, the index number in series starts from **0**.

Such labels can be used to access a specified value. For example,

```
import pandas as pd

# create a list
data = [10, 20, 30, 40, 50]

# create a series from the list
my_series = pd.Series(data)

# display third value in the series
print(my_series[2])
```

Run Code »

Output

```
30
```

Here, we accessed the third element of `my_series` using a label.

We can also specify labels while creating the series using the `index` argument in the `Series()` method. For example,

```
import pandas as pd

# create a list
a = [1, 3, 5]

# create a series and specify labels
my_series = pd.Series(a, index = ["x", "y", "z"])

print(my_series)
```

Run Code »

Output

```
x    1
y    3
z    5
dtype: int64
```

In this example,we passed `index = ["x", "y", "z"]` as an argument to `Series()` to specify the labels explicitly.

To access the series elements, we use the specified labels instead of the default index number. For example,

To access the series elements, we use the specified labels instead of the default index number. For example,

```
import pandas as pd

# create a list
a = [1, 3, 5]

# create a series and specify labels
my_series = pd.Series(a, index = ["x", "y", "z"])

# display the value with label y
print(my_series["y"])
```

Run Code »

Output

```
3
```

Create Series From a Python Dictionary

You can also create a Pandas Series from a Python dictionary. For example,

```
import pandas as pd

# create a dictionary
grades = {"Semester1": 3.25, "Semester2": 3.28, "Semester3": 3.75}

# create a series from the dictionary
my_series = pd.Series(grades)

# display the series
print(my_series)
```

Run Code »

Output

```
Semester1    3.25
Semester2    3.28
Semester3    3.75
dtype: float64
```

Here, we created a series named `my_series` of type `float64` with a dictionary named `grades`.

Notice that the **keys** of the dictionary have become the labels.

We can further customize the series by selecting specific items from the dictionary using the `index` argument. For example,

```
import pandas as pd

# create a dictionary
grades = {"Semester1": 3.25, "Semester2": 3.28, "Semester3": 3.75}

# select specific dictionary items using index argument
my_series = pd.Series(grades, index = ["Semester1", "Semester2"])

# display the series
print(my_series)
```

Run Code »

Output

```
Semester1    3.25
Semester2    3.28
dtype: float64
```

Pandas DataFrame

A DataFrame is like a table where the data is organized in rows and columns. It is a two-dimensional data structure like a two-dimensional array. For example,

```
     Country      Capital      Population
0    Canada       Ottawa       37742154
1    Australia    Canberra     25499884
2    UK           London       67886011
3    Brazil       Brasília     212559417
```

Here,

- `Country`, `Capital` and `Population` are the column names.
- Each row represents a record, with the index value on the left. The index values are auto-assigned starting from **0**.
- Each column contains data of the same type. For instance, `Country` and `Capital` contain strings, and `Population` contains integers.

The DataFrame is similar to a table in a SQL database, or a spreadsheet in Excel. It is designed to manage ordered and unordered datasets in Python.

Create a Pandas DataFrame

We can create a Pandas DataFrame in the following ways:

- Using Python Dictionary
- Using Python List
- From a File
- Creating an Empty DataFrame

Pandas DataFrame Using Python Dictionary

We can create a dataframe using a dictionary by passing it to the `DataFrame()` function. For example,

```
import pandas as pd

# create a dictionary
data = {'Name': ['John', 'Alice', 'Bob'],
        'Age': [25, 30, 35],
        'City': ['New York', 'London', 'Paris']}

# create a dataframe from the dictionary
df = pd.DataFrame(data)

print(df)
```

Run Code »

Output

```
    Name  Age      City
0   John   25  New York
1   Alice  30  London
2   Bob    35  Paris
```

In this example, we created a dictionary called `data` that contains the column names (`Name` , `Age` , `City`) as keys, and lists of values as their respective values.

We then used the `pd.DataFrame()` function to convert the dictionary into a DataFrame called `df` .

Pandas DataFrame Using Python List

We can also create a DataFrame using a two-dimensional list. For example,

```
import pandas as pd

# create a two-dimensional list
data = [['John', 25, 'New York'],
       ['Alice', 30, 'London'],
       ['Bob', 35, 'Paris']]

# create a DataFrame from the list
df = pd.DataFrame(data, columns=['Name', 'Age', 'City'])

print(df)
```

Run Code »

Output

```
    Name  Age      City
0   John   25  New York
1  Alice   30    London
2    Bob   35     Paris
```

In this example, we created a two-dimensional list called `data` containing nested lists.

The `DataFrame()` function converts the 2-D list to a DataFrame. Each nested list behaves like a row of data in the DataFrame.

The `columns` argument provides a name to each column of the DataFrame.

Pandas DataFrame From a File

Another common way to create a DataFrame is by loading data from a CSV (Comma-Separated Values) file. For example,

```
import pandas as pd

# load data from a CSV file
df = pd.read_csv('data.csv')

print(df)
```

In this example, we used the `read_csv()` function which reads the CSV file `data.csv`, and automatically creates a DataFrame object `df`, containing data from the CSV file.

> **Note:** We can also create a DataFrame using other file types like JSON, Excel spreadsheet, SQL database, etc. The methods to read different file types are listed below:
>
> - JSON - *`read_json()`*
> - Excel spreadsheet - `read_excel()`
> - SQL - *`read_sql()`*

Create an Empty DataFrame

Sometimes we may want to create an empty DataFrame and then add data later. For example,

```
import pandas as pd

# create an empty DataFrame
df = pd.DataFrame()

print(df)
```

Run Code »

Output

```
Empty DataFrame
Columns: []
Index: []
```

In this example, we have created an empty DataFrame by calling `pd.DataFrame()` without any arguments.

Here, both the `Columns` and `Index` lists are empty in the DataFrame.The DataFrame has no data, but it can be used as a container to store and manipulate data later.

Default Index

When we create a DataFrame or Series without specifying an index explicitly, Pandas assigns a default integer index starting from **0**. For example,

```
import pandas as pd

data = {'Name': ['John', 'Alice', 'Bob'],
        'Age': [25, 28, 32],
        'City': ['New York', 'London', 'Paris']}

df = pd.DataFrame(data)
print(df)
```

Run Code »

Output

```
    Name  Age      City
0   John   25  New York
1  Alice   28    London
2    Bob   32     Paris
```

In this example, the default index `[0, 1, 2]` is automatically assigned to the rows.

Setting Index

We can set an existing column as the index using the `set_index()` method. For example,

```
import pandas as pd

# create dataframe
data = {'Name': ['John', 'Alice', 'Bob'],
        'Age': [25, 28, 32],
        'City': ['New York', 'London', 'Paris']}

df = pd.DataFrame(data)

# set the 'Name' column as index
df.set_index('Name', inplace=True)

print(df)
```

Run Code »

Output

```
Name      Age    City
John      25  New York
Alice     28    London
Bob       32     Paris
```

In this example, the `Name` column is set as the index, replacing the default integer index.

Here, the `inplace=True` parameter performs the operation directly on the object itself, without creating a new object. When we specify `inplace=True`, the original object is modified, and the changes are directly applied.

Creating a Range Index

We can create a range index with specific start and end values using the `RangeIndex()` function. For example,

```
import pandas as pd

# create dataframe
data = {'Name': ['John', 'Alice', 'Bob'],
        'Age': [25, 28, 32],
        'City': ['New York', 'London', 'Paris']}

df = pd.DataFrame(data)

# create a range index
df = pd.DataFrame(data, index=pd.RangeIndex(5, 8, name='Index'))

print(df)
```

Run Code »

Output

```
         Name  Age      City
Index
5        John   25  New York
6       Alice   28    London
7         Bob   32     Paris
```

Here, a range index from **5** to **8**(excluded) is created with the name `Index`.

Modifying Indexes in Pandas

Pandas allows us to make changes to indexes easily. Some common modification operations are:

- Renaming Index
- Resetting Index

Renaming Index

We can rename an index using the `rename()` method. For example,

```
import pandas as pd

# create a dataframe
data = {'Name': ['John', 'Alice', 'Bob'],
        'Age': [25, 28, 32],
        'City': ['New York', 'London', 'Paris']}
df = pd.DataFrame(data)

# display original dataframe
print('Original DataFrame:')
print(df)
print()

# rename index
df.rename(index={0: 'A', 1: 'B', 2: 'C'}, inplace=True)

# display dataframe after index is renamed
print('Modified DataFrame')
print(df)
```

Run Code »

Output

```
Original DataFrame:
    Name  Age      City
0   John   25  New York
1  Alice   28    London
2    Bob   32     Paris

Modified DataFrame
    Name  Age      City
A   John   25  New York
B  Alice   28    London
C    Bob   32     Paris
```

In this example, we renamed the indexes **0**, **1**, and **2** to `'A'`, `'B'`, and `'C'` respectively.

Resetting Index

We can reset the index to the default integer index using the `reset_index()` method. For example,

```
import pandas as pd

data = {'Name': ['John', 'Alice', 'Bob'],
        'Age': [25, 28, 32],
        'City': ['New York', 'London', 'Paris']}

# create a dataframe
df = pd.DataFrame(data)

# rename index
df.rename(index={0: 'A', 1: 'B', 2: 'C'}, inplace=True)

# display dataframe
print('Original DataFrame:')
print(df)
print('\n')

# reset index
df.reset_index(inplace=True)

# display dataframe after index is reset
print('Modified DataFrame:')
print(df)
```

Run Code »

Output

```
Original DataFrame:
    Name  Age      City
A   John   25  New York
B  Alice   28    London
C    Bob   32     Paris

Modified DataFrame:
  index   Name  Age      City
0     A   John   25  New York
1     B  Alice   28    London
2     C    Bob   32     Paris
```

Access Rows by Index

We can access rows of a DataFrame using the `.iloc` property. For example,

```
import pandas as pd

# create a dataframe
data = {'Name': ['John', 'Alice', 'Bob'],
        'Age': [25, 28, 32],
        'City': ['New York', 'London', 'Paris']}

df = pd.DataFrame(data)

second_row = df.iloc[1]

print(second_row)
```

Run Code »

Output

```
Name     Alice
Age         28
City    London
Name: 1, dtype: object
```

In this example, we displayed the second row of the `df` DataFrame by its index value (**1**) using the `.iloc` property.

To learn more, please visit the Pandas Indexing and Slicing article.

Get DataFrame Index

We can access the DataFrame Index using the `index` attribute. For example,

```
import pandas as pd

# create a dataframe
data = {'Name': ['John', 'Alice', 'Bob'],
        'Age': [25, 28, 32],
        'City': ['New York', 'London', 'Paris']}

df = pd.DataFrame(data)

# return index object
print(df.index)

# return index values
print(df.index.values)
```

Run Code »

Create Series From Pandas Array

In Pandas, we can directly create Pandas Series from Pandas Array.

For that we use the `Series()` method. Let's look at an example.

```
import pandas as pd

# create a Pandas array
arr = pd.array([18, 20, 19, 21, 22])

# create a Pandas series from the Pandas array
arr_series = pd.Series(arr)

print(arr_series)
```

Run Code »

Output

```
0    18
1    20
2    19
3    21
4    22
dtype: Int64
```

Here, we have used `pd.Series(arr)` to create a Series from Pandas array named `arr`.

In the output,

1. The left column represents the index of the Series. The default index is a sequence of integers starting from **0**.

Pandas DataFrame Analysis

Pandas DataFrame objects come with a variety of built-in functions like `head()`, `tail()` and `info()` that allow us to view and analyze DataFrames.

View Data in a Pandas DataFrame

A Pandas Dataframe can be displayed as any other Python variable using the `print()` function.

However, when dealing with very large DataFrames with large numbers of rows and columns, the `print()` function is unable to display the whole DataFrame. Instead, it prints only a part of the DataFrame.

In the case of large DataFrames, we can use `head()`, `tail()` and `info()` methods to get the overview of the DataFrame.

Pandas head()

The `head()` method provides a rapid summary of a DataFrame. It returns the column headers and a specified number of rows from the beginning. For example,

```
import pandas as pd

# create a dataframe
data = {'Name': ['John', 'Alice', 'Bob', 'Emma', 'Mike', 'Sarah', 'David', 'Linda', 'Tom'
        'Age': [25, 30, 35, 28, 32, 27, 40, 33, 29, 31],
        'City': ['New York', 'Paris', 'London', 'Sydney', 'Tokyo', 'Berlin', 'Rome', 'Mad
df = pd.DataFrame(data)

# display the first three rows
print('First Three Rows:')
print(df.head(3))
print()
```

Pandas tail()

The `tail()` method is similar to `head()` but it returns data starting from the end of the DataFrame. For example,

```
import pandas as pd

# create a dataframe
data = {'Name': ['John', 'Alice', 'Bob', 'Emma', 'Mike', 'Sarah', 'David', 'Linda', 'Tom'
        'Age': [25, 30, 35, 28, 32, 27, 40, 33, 29, 31],
        'City': ['New York', 'Paris', 'London', 'Sydney', 'Tokyo', 'Berlin', 'Rome', 'Mad

df = pd.DataFrame(data)

# display the last three rows
print('Last Three Rows:')
print(df.tail(3))
print()

# display the last five rows
print('Last Five Rows:')
print(df.tail())
```

Run Code »

Output

```
Last Three Rows:
    Name   Age     City
7  Linda   33   Madrid
8    Tom   29  Toronto
9  Emily   31   Moscow

Last Five Rows:
    Name   Age     City
5  Sarah   27   Berlin
6  David   40     Rome
7  Linda   33   Madrid
8    Tom   29  Toronto
9  Emily   31   Moscow
```

Get DataFrame Information

The `info()` method gives us the overall information about the DataFrame such as its class, data type, size etc. For example,

```
import pandas as pd

# create dataframe
data = {'Name': ['John', 'Alice', 'Bob', 'Emma', 'Mike', 'Sarah', 'David', 'Linda', 'Tom'
        'Age': [25, 30, 35, 28, 32, 27, 40, 33, 29, 31],
        'City': ['New York', 'Paris', 'London', 'Sydney', 'Tokyo', 'Berlin', 'Rome', 'Mad
df = pd.DataFrame(data)

# get info about dataframe
df.info()
```

Run Code »

Output

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    10 non-null     object
 1   Age     10 non-null     int64
 2   City    10 non-null     object
dtypes: int64(1), object(2)
memory usage: 372.0+ bytes
```

As you can see, the `info()` method provides the following information about a Pandas DataFrame:

- **Class:** The class of the object, which indicates that it is a pandas DataFrame

- **RangeIndex**: The index range of the DataFrame, showing the starting and ending index values
- **Data columns**: The total number of columns in the DataFrame
- **Column names**: The names of the columns in the DataFrame
- **Non-Null Count**: The count of non-null values for each column
- **Dtype**: The data types of the columns
- **Memory usage**: The memory usage of the DataFrame in bytes

The provided information enables us to understand about the dataset like its structure, dimension, and missing values. This insight is essential for data exploration, cleaning, manipulation, and analysis.

Remove Rows/Columns from a Pandas DataFrame

We can use `drop()` to delete rows and columns from a DataFrame.

Example: Delete Rows

```
import pandas as pd

# create a sample DataFrame
data = {'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Felipe', 'Rita'],
        'Age': [25, 30, 35, 40, 22, 29],
        'City': ['New York', 'London', 'Paris', 'Tokyo', 'Bogota', 'Banglore']}
df = pd.DataFrame(data)

# display the original DataFrame
print("Original DataFrame:")
print(df)
print()

# delete row with index 4
df.drop(4, axis=0, inplace=True)

# delete row with index 5
df.drop(index=5, inplace=True)

# delete rows with index 1 and 3
df.drop([1, 3], axis=0, inplace=True)

# display the modified DataFrame after deleting rows
print("Modified DataFrame:")
print(df)
```

Run Code »

Output

```
Original DataFrame:
      Name  Age      City
0    Alice   25  New York
1      Bob   30    London
2  Charlie   35     Paris
3    David   40     Tokyo

Modified DataFrame:
      Name  Age      City
0    Alice   25  New York
2  Charlie   35     Paris
```

In this example, we deleted single rows using the `labels=4` and `index=5` parameters. We also deleted multiple rows with `labels=[1,3]` argument.

Here,

- `axis=0` : indicates that rows are to be deleted
- `inplace=True` : indicates that the changes are to be made in the original DataFrame

Example: Delete columns

```
import pandas as pd

# create a sample DataFrame
data = {'Name': ['Alice', 'Bob', 'Charlie', 'David'],
        'Age': [25, 30, 35, 40],
        'City': ['New York', 'London', 'Paris', 'Tokyo'],
        'Height': ['165', '178', '185', '171'],
        'Profession': ['Engineer', 'Entrepreneur', 'Unemployed', 'Actor'],
        'Marital Status': ['Single', 'Married', 'Divorced', 'Engaged']}
df = pd.DataFrame(data)

# display the original DataFrame
print("Original DataFrame:")
print(df)
print()

# delete age column
df.drop('Age', axis=1, inplace=True)

# delete marital status column
df.drop(columns='Marital Status', inplace=True)

# delete height and profession columns
df.drop(['Height', 'Profession'], axis=1, inplace=True)

# display the modified DataFrame after deleting rows
print("Modified DataFrame:")
```

Run Code »

Output

```
Original DataFrame:
      Name  Age      City   Height    Profession   Marital Status
0    Alice  25  New York    165      Engineer         Single
1      Bob  30    London    178  Entrepreneur        Married
2  Charlie  35     Paris    185    Unemployed       Divorced
3    David  40     Tokyo    171         Actor        Engaged

Modified DataFrame:
      Name      City
0    Alice  New York
1      Bob    London
2  Charlie     Paris
3    David     Tokyo
```

In this example, we deleted single columns using the `labels='Age'` and `columns='Marital Status'` parameters. We also deleted multiple columns with `labels=['Height', 'Profession']` argument.

Here,

- `axis=1` : indicates that columns are to be deleted
- `inplace=True` : indicates that the changes are to be made in the original DataFrame

Rename Labels in a DataFrame

We can rename columns in a Pandas DataFrame using the `rename()` function.

Example: Rename Columns

```
import pandas as pd

# create a sample DataFrame
data = {'Name': ['Alice', 'Bob', 'Charlie', 'David'],
        'Age': [25, 30, 35, 40],
        'City': ['New York', 'London', 'Paris', 'Tokyo']}
df = pd.DataFrame(data)

# display the original DataFrame
print("Original DataFrame:")
print(df)
print()

# rename column 'Name' to 'First_Name'
df.rename(columns= {'Name': 'First_Name'}, inplace=True)

# rename columns 'Age' and 'City'
df.rename(mapper= {'Age': 'Number', 'City':'Address'}, axis=1, inplace=True)

# display the DataFrame after renaming column
print("Modified DataFrame:")
print(df)
```

Run Code »

Output

```
Original DataFrame:
      Name  Age      City
0    Alice  25  New York
1      Bob  30    London
2  Charlie  35     Paris
3    David  40     Tokyo

Modified DataFrame:
  First_Name  Number   Address
0      Alice      25  New York
1        Bob      30    London
2    Charlie      35     Paris
3      David      40     Tokyo
```

In this example, we renamed a single column using the `columns={'Name': 'First_Name'}` parameter. We also renamed multiple columns with `mapper={'Age': 'Number', 'City':'Address'}` argument.

Here,

- `axis=1` : indicates that columns are to be renamed
- `inplace=True` : indicates that the changes are to be made in the original DataFrame

Example: Rename Row Labels

```
import pandas as pd

# create a sample DataFrame
data = {'Name': ['Alice', 'Bob', 'Charlie', 'David'],
        'Age': [25, 30, 35, 40],
        'City': ['New York', 'London', 'Paris', 'Tokyo']}
df = pd.DataFrame(data)

# display the original DataFrame
print("Original DataFrame:")
print(df)
print()

# rename column one index label
df.rename(index={0: 7}, inplace=True)

# rename columns multiple index labels
df.rename(mapper={1: 10, 2: 100}, axis=0, inplace=True)

# display the DataFrame after renaming column
print("Modified DataFrame:")
print(df)
```

Run Code »

Output

```
Original DataFrame:
      Name  Age      City
0    Alice   25  New York
1      Bob   30    London
2  Charlie   35     Paris
3    David   40     Tokyo

Modified DataFrame:
        Name  Age      City
7      Alice   25  New York
10       Bob   30    London
100  Charlie   35     Paris
3      David   40     Tokyo
```

Access Columns of a DataFrame

We can access columns of a DataFrame using the bracket (`[]`) operator. For example,

```
import pandas as pd

# create a DataFrame
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Age': [25, 32, 18, 47, 33],
    'City': ['New York', 'Paris', 'London', 'Tokyo', 'Sydney']
}
df = pd.DataFrame(data)

# access the Name column
names = df['Name']

print(names)
```

Run Code »

Output

```
0      Alice
1        Bob
2    Charlie
3      David
4        Eve
Name: Name, dtype: object
```

In this example, we accessed the `Name` column of `df` using the `[]` operator. It returned a series containing the values of the `Name` column.

We can also access multiple columns using the `[]` operator. For example,

```
import pandas as pd

# create a DataFrame
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Age': [25, 32, 18, 47, 33],
    'City': ['New York', 'Paris', 'London', 'Tokyo', 'Sydney']
}
df = pd.DataFrame(data)

# access multiple columns
name_city = df[['Name','City']]

print(name_city)
```

Run Code »

Output

```
      Name      City
0    Alice  New York
1      Bob     Paris
2  Charlie    London
3    David     Tokyo
4      Eve    Sydney
```

Select Data Using Indexing and Slicing

In Pandas, we can use square brackets and their labels or positions to select the data we want.

Let's look at an example.

import pandas as pd

 # create a DataFrame
data = {
'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],

```
        'Age': [25, 30, 22, 27, 29],
        'Salary': [50000, 60000, 45000, 55000, 52000]
}

df = pd.DataFrame(data)

# selecting a single column
name_column = df['Name']

print("Selecting single column: Name")
print(name_column)
print()

# selecting multiple columns
age_salary_columns = df[['Age', 'Salary']]

print("Selecting multiple columns: Age and Salary")
print(age_salary_columns.to_string(index=False))
print()

# selecting rows using slicing
selected_rows = df[1:4]

print("Selecting rows 1 to 3")
print(selected_rows.to_string(index=False))
print()
```

Output

```
Selecting single column: Name
0      Alice
1        Bob
2    Charlie
3      David
4        Eve
Name: Name, dtype: object

Selecting multiple columns: Age and Salary
Age  Salary
25   50000
30   60000
22   45000
27   55000
29   52000

Selecting rows 1 to 3
Name      Age  Salary
Bob        30    60000
Charlie    22    45000
David      27    55000
```

Using loc and iloc to Select Data

The `loc` and `iloc` methods in Pandas are used to access data by using label or integer index.

1. `loc` selects rows and columns with specific labels
2. `iloc` selects rows and columns at specific index

Let's take a look at an example.

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Emily'],
    'Age': [25, 30, 22, 27, 29],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston', 'San Francisco']
}

df = pd.DataFrame(data)
print(f"Original DataFrame \n {df} \n")

# loc to select rows and columns by labels
# select rows 1 to 3 and columns Name and Age
selected_data_loc = df.loc[1:3, ['Name', 'Age']]

print(selected_data_loc.to_string(index = False))
print()

# iloc to select rows and columns by index
# select rows 1 to 3 and columns 0 and 2
selected_data_iloc = df.iloc[1:4, [0, 2]]

print(selected_data_iloc.to_string(index = False))
```

Output

```
Original DataFrame
      Name  Age           City
0    Alice   25       New York
1      Bob   30    Los Angeles
2  Charlie   22        Chicago
3    David   27        Houston
4    Emily   29  San Francisco

Name     Age
Bob       30
Charlie   22
David     27

Name        City
Bob      Los Angeles
Charlie  Chicago
David    Houston
```

Here,

- Using `df.loc[1:3, ['Name', 'Age']]` - selects rows **1** to **3** and columns `Name` and `Age` from `df`
- Using `df.iloc[1:4, [0, 2]]` - selects rows **1** to **3** and columns at index positions **0** and **2** from `df`

Select Rows Based on Specific Criteria

In Pandas, we can use boolean conditions to filter rows based on specific criteria. For example,

```
import pandas as pd

# creating a DataFrame
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Emily'],
    'Age': [25, 30, 22, 28, 24],
    'Gender': ['Female', 'Male', 'Male', 'Male', 'Female']
}

df = pd.DataFrame(data)

# select rows where Age is greater than 25
selected_rows = df[df['Age'] > 25]

print(selected_rows)
```

Run Code »

Output

```
     Name  Age Gender
1     Bob   30   Male
3   David   28   Male
```

query() to Select Data

The `query()` method in Pandas allows you to select data using a more SQL-like syntax.

Let's take a look at an example.

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eva'],
    'Age': [25, 30, 22, 28, 35],
    'Score': [85, 90, 75, 80, 95]
}

df = pd.DataFrame(data)

# select the rows where the age is greater than 25
selected_rows = df.query('Age > 25')

print(selected_rows.to_string(index = False))
```

Run Code »

Output

```
Name  Age  Score
Bob    30     90
David  28     80
Eva    35     95
```

Select Rows Based on a List of Values

Pandas provides us with the method named `isin()` to filter rows based on a list of values. For example,

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Emily'],
    'Age': [25, 30, 22, 28, 24]
}

df = pd.DataFrame(data)

# create a list of names to select
names_to_filter = ['Bob', 'David']

# use isin() to select rows based on the 'Name' column
selected_rows = df[df['Name'].isin(names_to_filter)]

print(selected_rows.to_string(index = False))
```

Run Code »

Output

```
Name  Age
Bob   30
David 28
```

Find Duplicate Entries

We can find duplicate entries in a DataFrame using the `duplicated()` method. It returns `True` if a row is duplicated and returns `False` otherwise.

```
import pandas as pd

# create dataframe
data = {
    'Name': ['John', 'Anna', 'John', 'Anna', 'John'],
    'Age': [28, 24, 28, 24, 19],
    'City': ['New York', 'Los Angeles', 'New York', 'Los Angeles', 'Chicago']
}
df = pd.DataFrame(data)

# check for duplicate entries
print(df.duplicated())
```

Run Code »

Output

```
0    False
1    False
2     True
3     True
4    False
dtype: bool
```

Example: Find Duplicates Based on Columns

By default, `duplicated()` considers all columns. To find duplicates based on certain columns, we can pass them as a list to the `duplicated()` function.

```
import pandas as pd

# create dataframe
data = {
    'Name': ['John', 'Anna', 'Johnny', 'Anna', 'John'],
    'Age': [28, 24, 28, 24, 19],
    'City': ['New York', 'Las Vegas', 'New York', 'Los Angeles', 'Chicago']
}
df = pd.DataFrame(data)

# check for duplicate entries in columns Name and Age
print(df.duplicated(subset=['Name', 'Age']))
```

Run Code »

Output

```
0    False
1    False
2    False
3     True
4    False
dtype: bool
```

Remove Duplicate Entries

We can remove duplicate entries in Pandas using the `drop_duplicates()` method. For example,

```
import pandas as pd

# create dataframe
data = {
    'Name': ['John', 'Anna', 'John', 'Anna', 'John'],
    'Age': [28, 24, 28, 24, 19],
    'City': ['New York', 'Los Angeles', 'New York', 'Los Angeles', 'Chicago']
}
df = pd.DataFrame(data)

# remove duplicates
df.drop_duplicates(inplace=True)

print(df)
```

Run Code »

Output

```
   Name  Age         City
0  John   28     New York
1  Anna   24  Los Angeles
4  John   19      Chicago
```

```python
import pandas as pd

# create dataframe
data = {
    'Name': ['John', 'Anna', 'John', 'Anna', 'John'],
    'Age': [28, 24, 28, 24, 19],
    'City': ['New York', 'Los Angeles', 'New York', 'Los Angeles', 'Chicago']
}
df = pd.DataFrame(data)

# remove duplicates, keep last entries
df.drop_duplicates(keep='last', inplace=True)

print(df)
```

Run Code »

Output

```
   Name  Age         City
2  John   28     New York
3  Anna   24  Los Angeles
4  John   19      Chicago
```

Pandas Pivot

The `pivot()` function in Pandas reshapes data based on column values. It takes simple column-wise data as input, and groups the entries into a two-dimensional table.

Date	City	Temperature
2023-01-01	New York	32
2023-01-01	Los Angeles	75
2023-01-02	New York	30
2023-01-02	Los Angeles	77

pivot →

	Temperature	
City	Los Angeles	New York
Date		
2023-01-01	75	32
2023-01-02	77	30

Pivot Operation in Pandas

Let's look at an example.

Let's look at an example.

```
import pandas as pd

# create a dataframe
data = {'Date': ['2023-01-01', '2023-01-01', '2023-01-02', '2023-01-02'],
        'City': ['New York', 'Los Angeles', 'New York', 'Los Angeles'],
        'Temperature': [32, 75, 30, 77]}
df = pd.DataFrame(data)

print("Original DataFrame\n", df)
print()

# pivot the dataframe
pivot_df = df.pivot(index='Date', columns='City', values='Temperature')

print("Reshaped DataFrame\n", pivot_df)
```

Run Code »

Output

```
Original DataFrame
         Date         City  Temperature
0  2023-01-01     New York           32
1  2023-01-01  Los Angeles           75
2  2023-01-02     New York           30
3  2023-01-02  Los Angeles           77

Reshaped DataFrame
City        Los Angeles  New York
Date
2023-01-01           75        32
2023-01-02           77        30
```

Example: pivot() for Multiple Values

If we omit the `values` argument in `pivot()`, it selects all the remaining columns (besides the ones specified `index` and `columns`) as values for the pivot table.

Let's see an example.

```
import pandas as pd

# create a dataframe
data = {'Date': ['2023-01-01', '2023-01-01', '2023-01-02', '2023-01-02'],
        'City': ['New York', 'Los Angeles', 'New York', 'Los Angeles'],
        'Temperature': [32, 75, 30, 77],
        'Humidity': [80, 10, 85, 5]}

df = pd.DataFrame(data)

print('Original DataFrame')
print(df)
print()

# pivot the dataframe
pivot_df = df.pivot(index='Date', columns='City')

print('Reshaped DataFrame')
print(pivot_df)
```

Output

```
Original DataFrame
         Date         City  Temperature  Humidity
0  2023-01-01     New York           32        80
1  2023-01-01  Los Angeles           75        10
2  2023-01-02     New York           30        85
3  2023-01-02  Los Angeles           77         5

Reshaped DataFrame
           Temperature             Humidity
City       Los Angeles New York Los Angeles New York
Date
2023-01-01          75       32          10       80
2023-01-02          77       30           5       85
```

In this example, we created a pivot table for multiple values i.e. `Temperature` and `Humidity`.

Write to CSV Files

We used `read_csv()` to read data from a CSV file into a DataFrame.

Pandas also provides the `to_csv()` function to write data from a DataFrame into a CSV file.

Let's see an example.

```
import pandas as pd

# create a dictionary
data = {'Name': ['John', 'Alice', 'Bob'],
        'Age': [25, 30, 35],
        'City': ['New York', 'London', 'Paris']}

# create a dataframe from the dictionary
df = pd.DataFrame(data)

# write dataframe to csv file
df.to_csv('output.csv', index=False)
```

Output

```
Name,Age,City
John,25,New York
Alice,30,London
Bob,35,Paris
```

Example: to_csv() With Arguments

```
import pandas as pd

# create dataframe
data = {'Name': ['Tom', 'Nick', 'John', 'Tom'],
        'Age': [20, 21, 19, 18],
        'City': ['New York', 'London', 'Paris', 'Berlin']}
df = pd.DataFrame(data)

# write to csv file
df.to_csv('output.csv', sep = ';', index = False, header = True)
```

Output

```
Name;Age;City
Tom;20;New York
Nick;21;London
John;19;Paris
Tom;18;Berlin
```

In this example, we wrote a DataFrame to the CSV file `'output.csv'` using the `to_csv()` method. We used some arguments to write necessary data to the file in required format.

Here,

- `sep = ';'` : specifies the delimiter as `';'`
- `index = False` : instructs not to include the index column in the output file
- `header = True` : instructs to include the column names as the header in the output file

Pandas JSON

Pandas offers methods like `read_json()` and `to_json()` to work with JSON (JavaScript Object Notation) data.

JSON is a plain text document that follows a format similar to a JavaScript object. It consists of key-value pairs, where the keys are strings and the values can be strings, numbers, booleans, arrays, or even other JSON objects.

Here's an example of a JSON.

```
[
  {
    "name": "John",
    "age": 30,
    "city": "New York"
  },
  {
    "name": "Emily",
    "age": 28,
    "city": "San Francisco"
  },
  {
    "name": "David",
    "age": 35,
    "city": "Chicago"
  }
]
```

Let's name this JSON file `data.json`.

Read JSON in Pandas

To read JSON data into a Pandas DataFrame, you can use the `read_json()` function.

Let's read the JSON file `data.json` we created before.

Write JSON in Pandas

To write a Pandas DataFrame to a JSON file, you can use the `to_json()` function. For example,

```
import pandas as pd

# create a dictionary
data = {'Name': ['John', 'Alice', 'Bob'],
        'Age': [25, 30, 35],
        'City': ['New York', 'London', 'Paris']}

# create a dataframe from the dictionary
df = pd.DataFrame(data)

# write dataframe to json file
df.to_json('output.json')
```

Output

```
{"Name":{"0":"John","1":"Alice","2":"Bob"},"Age":{"0":25,"1":30,"2":35},"City":{"0":"New
```

The above code snippet writes the `df` DataFrame to the JSON file `output.json`.

Example: Write JSON

```
import pandas as pd

# create a dictionary
data = {'Name': ['John', 'Alice', 'Bob'],
             'Age': [25, 30, 35],
             'City': ['New York', 'London', 'Paris']}

# create a dataframe from the dictionary
df = pd.DataFrame(data)

# write dataframe to json file
df.to_json('output.json', orient = 'records', indent = 4)
```

Output

```
[
    {
        "Name":"John",
        "Age":25,
        "City":"New York"
    },
    {
        "Name":"Alice",
        "Age":30,
        "City":"London"
    },
    {
        "Name":"Bob",
        "Age":35,
        "City":"Paris"
    }
]
```

In this example, we exported the DataFrame `df` to the `output.json` file.

Here,

- `orient = 'records'` : represents each row in the DataFrame as a JSON object
- `indent = 4` : sets the number of spaces used for indentation to **4**

```
import matplotlib.pyplot as plt
```

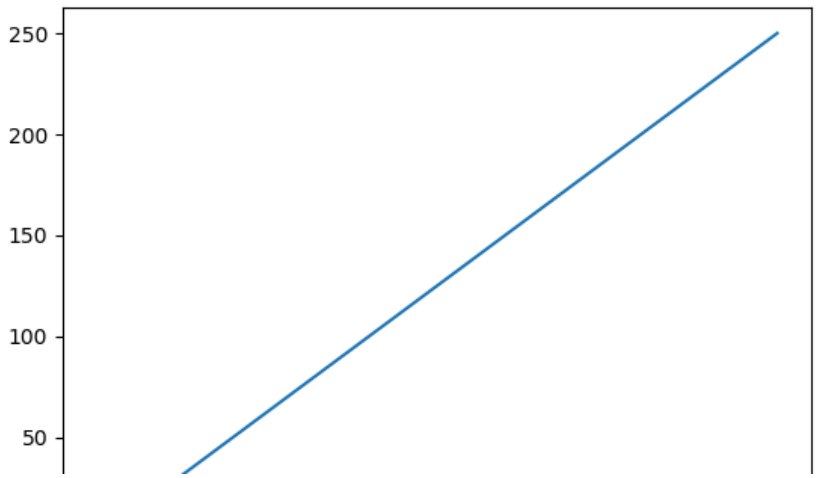
Now the Pyplot package can be referred to as `plt`.

Example

Draw a line in a diagram from position (0,0) to position (6,250):

```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([0, 6])
ypoints = np.array([0, 250])

plt.plot(xpoints, ypoints)
plt.show()
```

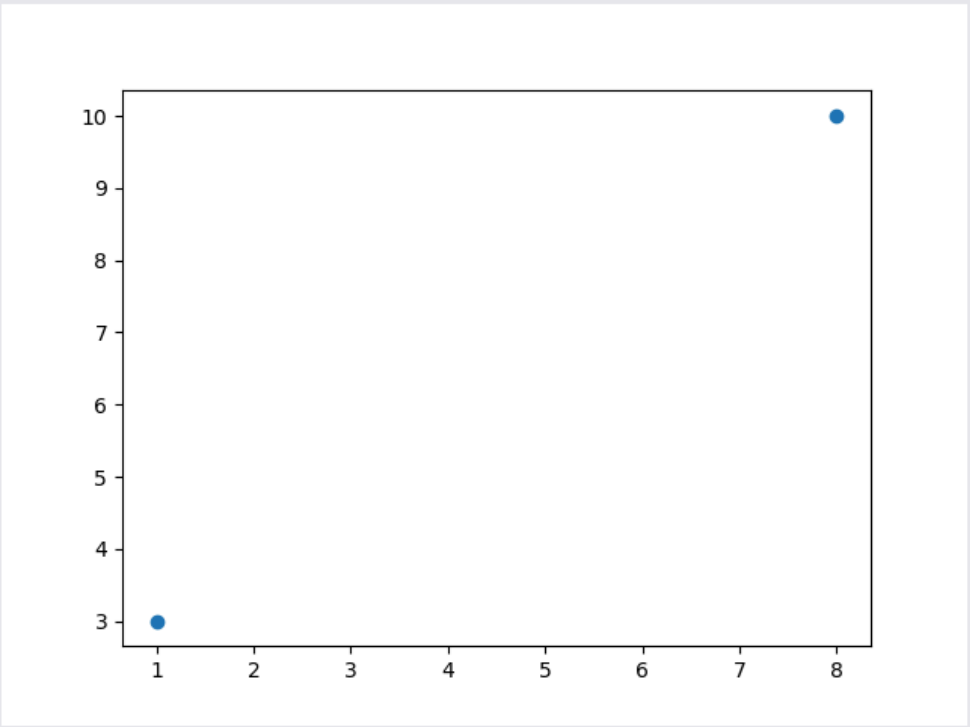
Result:



Draw two points in the diagram, one at position (1, 3) and one in position (8, 10):

```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([1, 8])
ypoints = np.array([3, 10])

plt.plot(xpoints, ypoints, 'o')
plt.show()
```

Result:



Try it Yourself »

Multiple Points

You can plot as many points as you like, just make sure you have the same number of points in both axis.

Example

Draw a line in a diagram from position (1, 3) to (2, 8) then to (6, 1) and finally to position (8, 10):

```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([1, 2, 6, 8])
ypoints = np.array([3, 8, 1, 10])

plt.plot(xpoints, ypoints)
plt.show()
```

Result:

Default X-Points

If we do not specify the points on the x-axis, they will get the default values 0, 1, 2, 3 etc., depending on the length of the y-points.

So, if we take the same example as above, and leave out the x-points, the diagram will look like this:

Example

Plotting without x-points:

```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3, 8, 1, 10, 5, 7])

plt.plot(ypoints)
plt.show()
```

Result:

Markers

You can use the keyword argument `marker` to emphasize each point with a specified marker:

Example

Mark each point with a circle:

```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3, 8, 1, 10])

plt.plot(ypoints, marker = 'o')
plt.show()
```

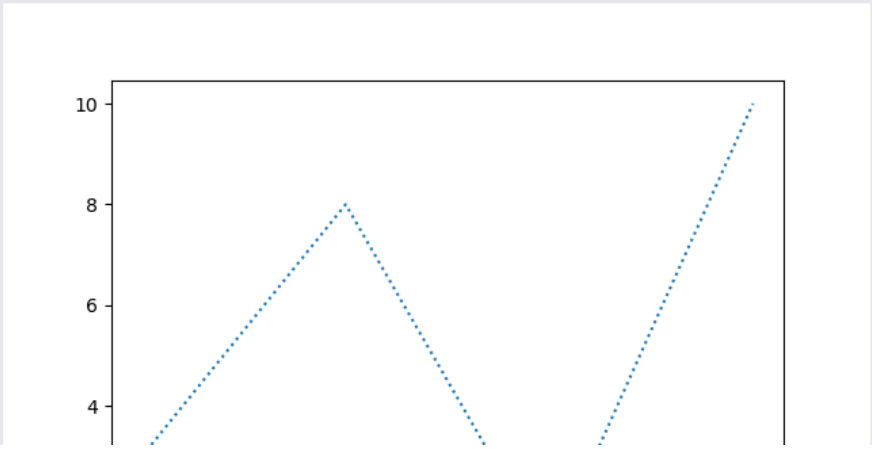
Result:

Example

Use a dotted line:

```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3, 8, 1, 10])

plt.plot(ypoints, linestyle = 'dotted')
plt.show()
```

Result:



Create Labels for a Plot

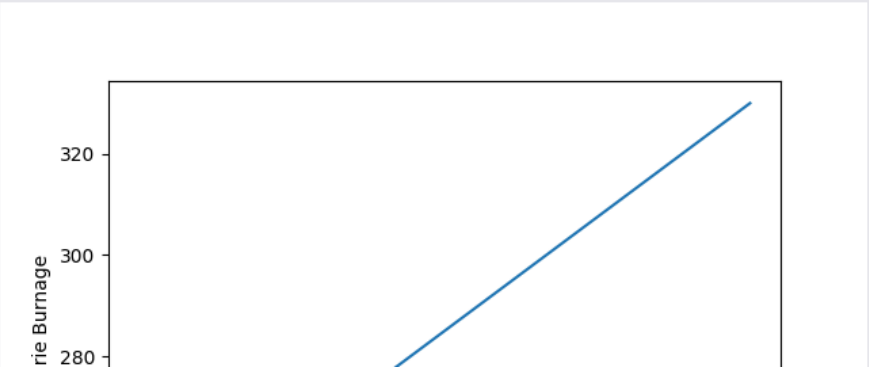
With Pyplot, you can use the `xlabel()` and `ylabel()` functions to set a label for the x- and y-axis.

Example

Add labels to the x- and y-axis:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.plot(x, y)

plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.show()
```

Result:



Position the Title

You can use the `loc` parameter in `title()` to position the title.

Legal values are: 'left', 'right', and 'center'. Default value is 'center'.

Example

Position the title to the left:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.title("Sports Watch Data", loc = 'left')
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.plot(x, y)
plt.show()
```

Result:

Set Line Properties for the Grid

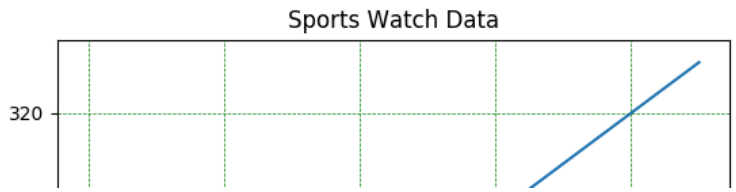
You can also set the line properties of the grid, like this: grid(color = '*color*', linestyle = '*linestyle*', linewidth = *number*).

Example

Set the line properties of the grid:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.title("Sports Watch Data")
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.plot(x, y)

plt.grid(color = 'green', linestyle = '--', linewidth = 0.5)

plt.show()
```

Result:



Display Multiple Plots

With the `subplot()` function you can draw multiple plots in one figure:

Example

Draw 2 plots:

```
import matplotlib.pyplot as plt
import numpy as np

#plot 1:
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])

plt.subplot(1, 2, 1)
plt.plot(x,y)

#plot 2:
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])

plt.subplot(1, 2, 2)
plt.plot(x,y)

plt.show()
```

Result:

Colors

You can set your own color for each scatter plot with the `color` or the `c` argument:

Example

Set your own color of the markers:

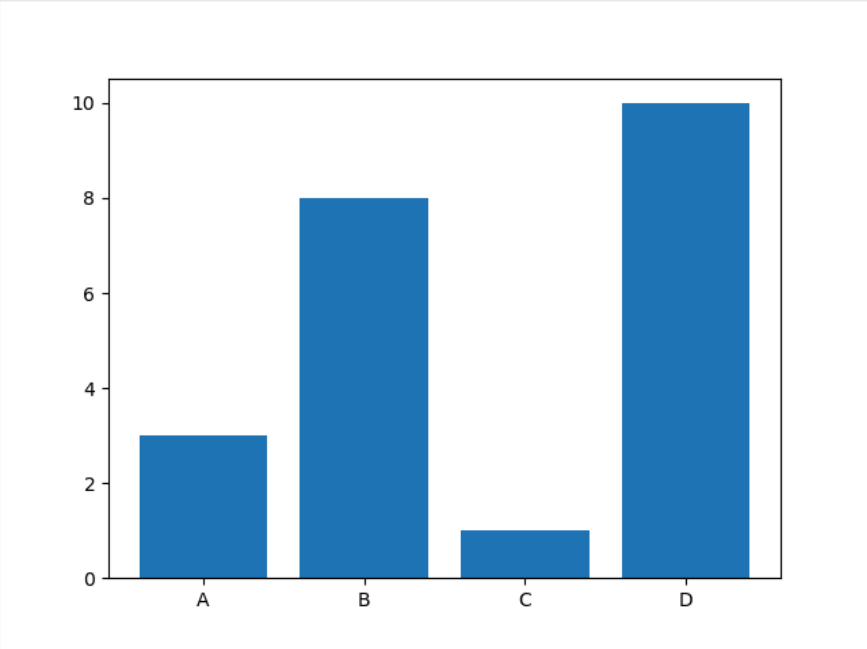
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
plt.scatter(x, y, color = 'hotpink')

x = np.array([2,2,8,1,15,8,12,9,7,3,11,4,7,14,12])
y = np.array([100,105,84,105,90,99,90,95,94,100,79,112,91,80,85])
plt.scatter(x, y, color = '#88c999')

plt.show()
```

Result:

Draw 4 bars:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.bar(x,y)
plt.show()
```

Result:



Horizontal Bars

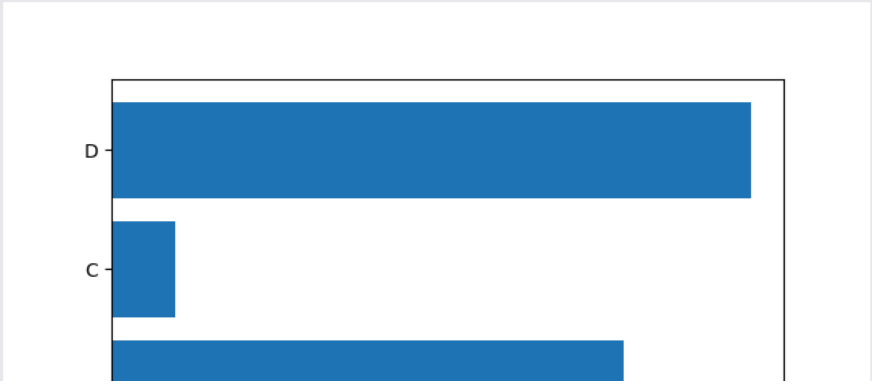
If you want the bars to be displayed horizontally instead of vertically, use the `barh()` function:

Example

Draw 4 horizontal bars:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.barh(x, y)
plt.show()
```

Result:



Color Hex

Or you can use Hexadecimal color values:

Example

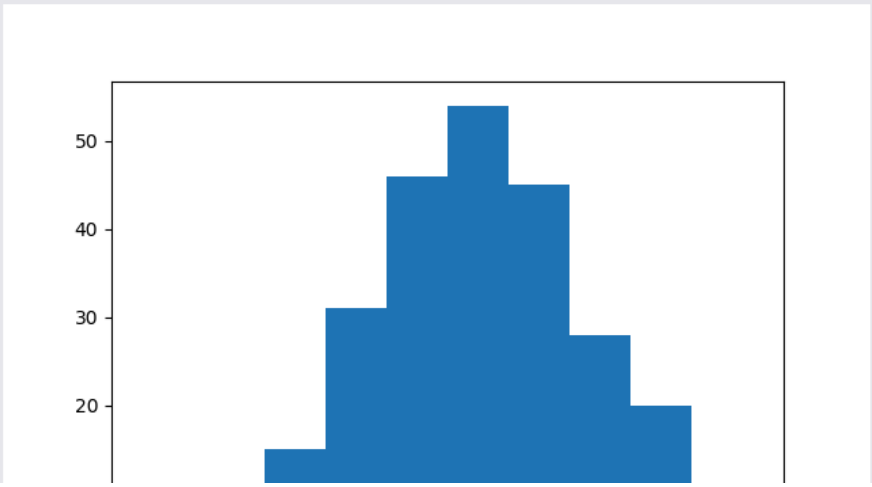
Draw 4 bars with a beautiful green color:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.bar(x, y, color = "#4CAF50")
plt.show()
```

Result:

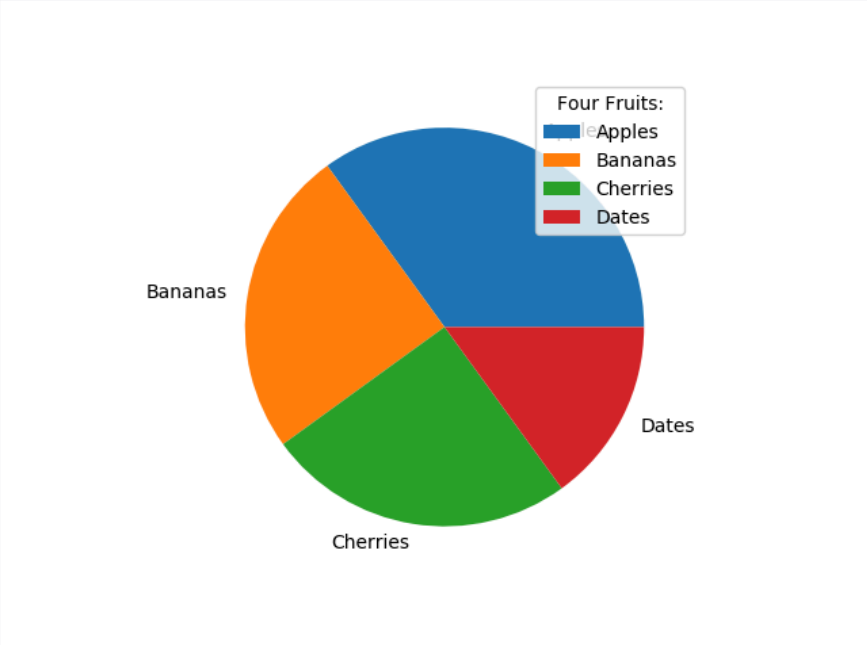
```
import matplotlib.pyplot as plt
import numpy as np

x = np.random.normal(170, 10, 250)

plt.hist(x)
plt.show()
```

Result:



```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]

plt.pie(y, labels = mylabels)
plt.legend(title = "Four Fruits:")
plt.show()
```

Result:



2. Scatter Plot

Scatter plots are used to visualize the relationship between two numerical variables. They help identify correlations or patterns. It can draw a two-dimensional graph.

Syntax: *seaborn.scatterplot(x=None, y=None)*

Parameters:
x, y: *Input data variables that should be numeric.*

Returns: *This method returns the Axes object with the plot drawn onto it.*

Let's visualize the data with a scatter plot and pandas:

Python

```
import pandas as pd
import seaborn as sns

# initialise data of lists
data = {'Name':[ 'Mohe' , 'Karnal' , 'Yrik' , 'jack' ],
        'Age':[ 30 , 21 , 29 , 28 ]}
df = pd.DataFrame( data )

seaborn.scatterplot(data['Age'],data['Weight'])
```

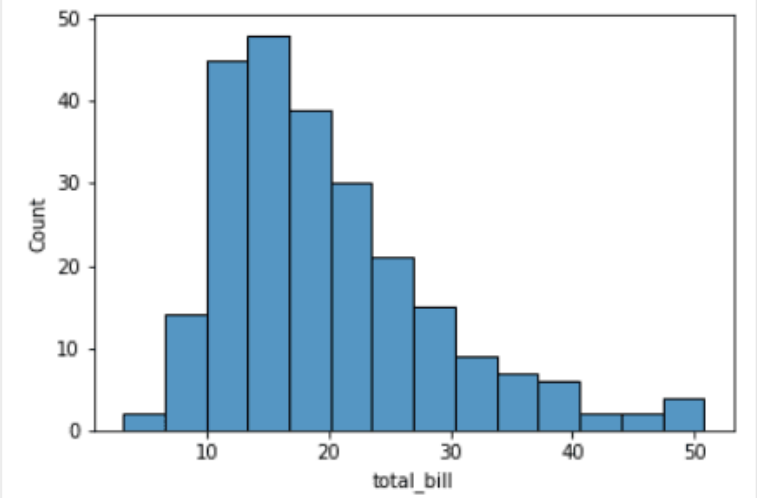
Output:

```
import seaborn as sns



# Load the Tips dataset

tips = sns.load_dataset("tips")



# Create a histogram of the total bill amounts

sns.histplot(data=tips, x="total_bill")
```

Explain code

POWERED BY datalab

Output:



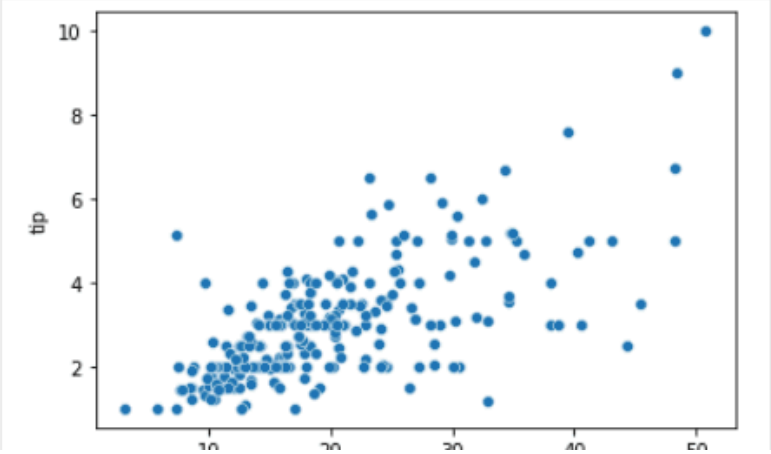
Seaborn scatter plots

Scatter plots are used to visualize the relationship between two continuous variables. Each point on the plot represents a single data point, and the position of the point on the x and y-axis represents the values of the two variables.

The plot can be customized with different colors and markers to help distinguish different groups of data points. In Seaborn, scatter plots can be created using the `scatterplot()` function.

```
import seaborn as sns

tips = sns.load_dataset("tips")

sns.scatterplot(x="total_bill", y="tip", data=tips)
```


Output:



Seaborn line plots

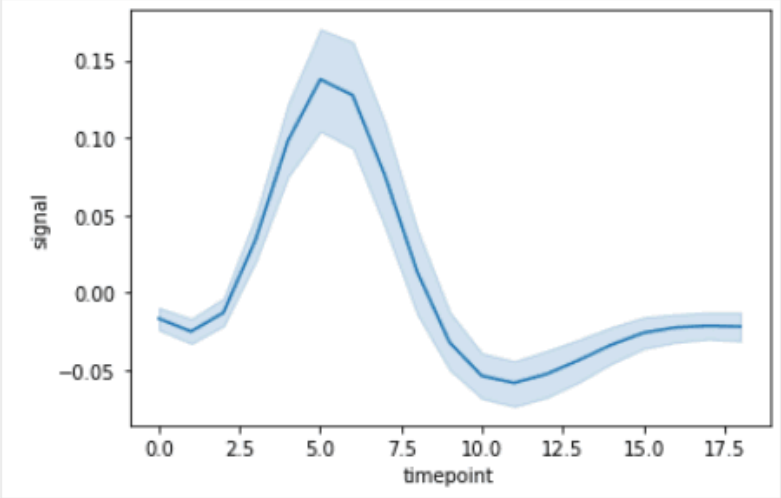
Line plots are used to visualize trends in data over time or other continuous variables. In a line plot, each data point is connected by a line, creating a smooth curve. In Seaborn, line plots can be created using the `lineplot()` function.

```
import seaborn as sns

fmri = sns.load_dataset("fmri")

sns.lineplot(x="timepoint", y="signal", data=fmri)
```

Explain code

POWERED BY datalab

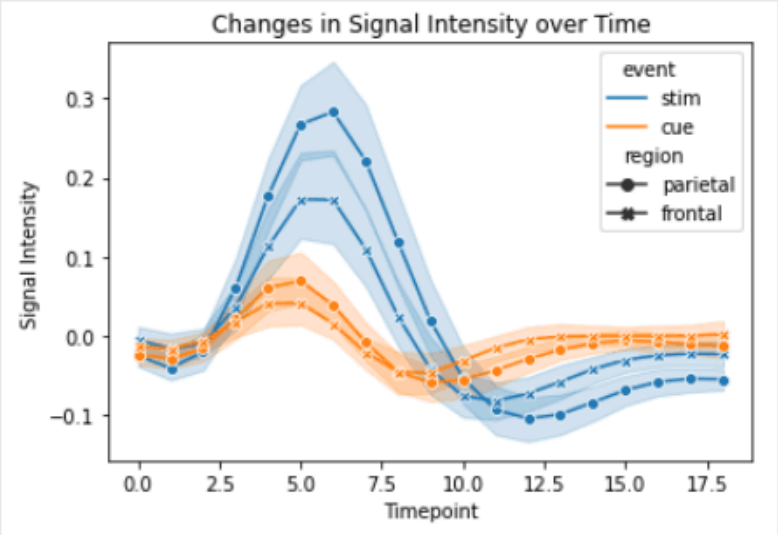
Output:



import seaborn as sns
import matplotlib.pyplot as plt
fmri = sns.load_dataset("fmri")
 # customize the line plot
sns.lineplot(x="timepoint", y="signal", hue="event", style="region", markers=True, dashes=False, data=fmri)

 # add labels and title
plt.xlabel("Timepoint")

```
plt.ylabel("Signal Intensity")
plt.title("Changes in Signal Intensity over Time")
# display the plot
plt.show()
```



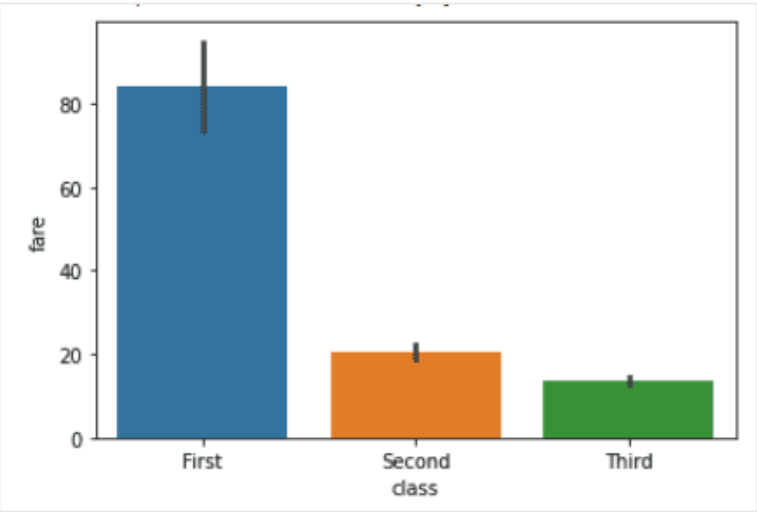
Seaborn bar plots

Bar plots are used to visualize the relationship between a categorical variable and a continuous variable. In a bar plot, each bar represents the mean or median (or any aggregation) of the continuous variable for each category. In Seaborn, bar plots can be created using the `barplot()` function.

```
import seaborn as sns

titanic = sns.load_dataset("titanic")

sns.barplot(x="class", y="fare", data=titanic)
```

Explain code

Output:



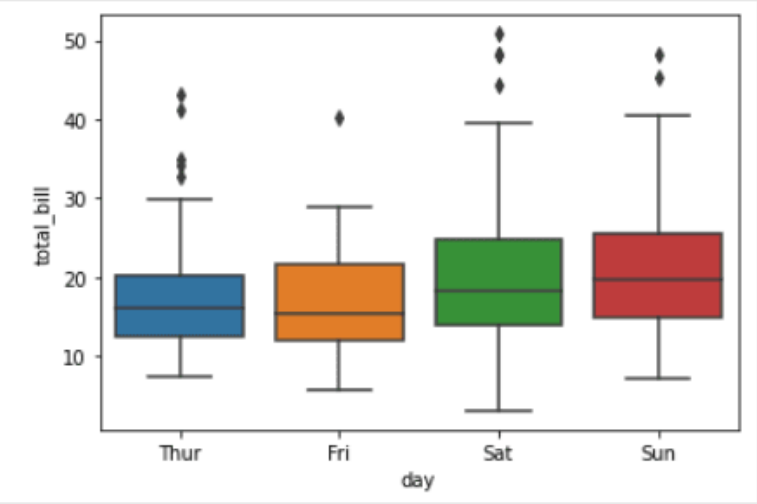
Seaborn box plots

Box plots are a type of visualization that shows the distribution of a dataset. They are commonly used to compare the distribution of one or more variables across different categories.

```
import seaborn as sns

tips = sns.load_dataset("tips")

sns.boxplot(x="day", y="total_bill", data=tips)
```

Explain code

Output:



Seaborn heatmaps

A heatmap is a graphical representation of data that uses colors to depict the value of a variable in a two-dimensional space. Heatmaps are commonly used to visualize the correlation between different variables in a dataset.

```
import seaborn as sns

import matplotlib.pyplot as plt



# Load the dataset

tips = sns.load_dataset('tips')



# Create a heatmap of the correlation between variables

corr = tips.corr()

sns.heatmap(corr)



# Show the plot

plt.show()
```

Seaborn pair plots

Pair plots are a type of visualization in which multiple pairwise scatter plots are displayed in a matrix format. Each scatter plot shows the relationship between two variables, while the diagonal plots show the distribution of the individual variables.

```
import seaborn as sns



# Load iris dataset

iris = sns.load_dataset("iris")



# Create pair plot

sns.pairplot(data=iris)



# Show plot

plt.show()
```